LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 9: HMVC, Authorization, dan AJAX
dengan React pada Laravel



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S
NIM: 707012530010
Kelas: D4SIKC-49-04
Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

A. Tujuan

Praktikum ini saya kerjakan untuk membangun aplikasi penerimaan mahasiswa baru berbasis arsitektur HMVC dengan React Starter Kit pada project pertemuan9_hmvc_react. Berbeda dengan Modul 8 yang memakai Blade dan jQuery, seluruh halaman modul pada praktikum ini ditulis dengan React dan dihubungkan lewat Inertia.js. Modul UserManagement bekerja dengan CRUD Inertia, sedangkan modul Pendaftaran bekerja dengan CRUD AJAX dan upload berkas. Akses setiap halaman dibatasi oleh role admin dan user yang diperiksa middleware. Tujuan yang ingin dicapai adalah:

1. Menyiapkan project Laravel React Starter Kit dan memasang paket nwidart/laravel-modules beserta merge-plugin pada composer.json.

2. Mengatur agar Inertia dapat menemukan halaman React di dalam folder modul melalui fungsi resolve pada app.tsx, serta mendaftarkan folder modul pada Tailwind CSS dan tsconfig.json.

3. Menambahkan kolom role pada tabel users, enum UserRole, factory, dan seeder, lalu menyesuaikan CreateNewUser Fortify agar akun hasil register selalu berperan user.

4. Membagikan data auth dan flash ke seluruh halaman React sebagai shared props, lalu membuat hook useSharedProps serta komponen AlertMessage, FlashMessage, dan ConfirmDialog.

5. Membuat middleware role, mendaftarkan aliasnya pada bootstrap/app.php, dan menampilkan menu sidebar sesuai role.

6. Membuat modul UserManagement dengan CRUD Inertia khusus admin, lengkap dengan pencarian, filter role, pagination, dan aturan admin tidak dapat menurunkan role atau menghapus akunnya sendiri.

7. Membuat modul Pendaftaran dengan CRUD AJAX memakai fetch dan FormData, termasuk upload berkas yang dapat dibuka kembali, diganti saat edit, dan ikut terhapus dari storage.

8. Menampilkan error validasi 422 di bawah setiap input pada form AJAX, serta pesan berhasil setelah data ditambah, diubah, atau dihapus.

9. Menguji seluruh fitur melalui browser dan pengujian otomatis Pest.

B. Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

● Laptop dengan prosesor 13th Gen Intel Core i7-13620H.

● RAM 16 GB (terbaca 15,7 GB).

● Sistem operasi Windows 11 Home Single Language build 10.0.26200.

Perangkat Lunak

● XAMPP 8.2.12, berisi Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.

● Composer 2.10.3 sebagai dependency manager PHP.

● Laravel Framework 12.71.0 dengan React Starter Kit (React 19, Inertia.js 2.0, Tailwind CSS 4, shadcn/ui, dan Laravel Fortify 1.41).

● nwidart/laravel-modules 12.0.5 sebagai paket arsitektur HMVC.

● Node.js 24.11.1 dan npm 11.6.2 untuk mengompilasi halaman React melalui Vite 6.

● Pest 3.8 sebagai framework pengujian otomatis.

● Visual Studio Code sebagai editor teks.

● Google Chrome sebagai browser dan alat pemeriksaan tab Network.

● Git untuk version control.

● Database MariaDB bernama hmvc_react.

C. Langkah-Langkah Praktikum

Aplikasi saya bangun di project pertemuan9_hmvc_react dengan database hmvc_react. Autentikasi memakai fitur bawaan React Starter Kit melalui Laravel Fortify, sedangkan dua modul HMVC diisi sesuai tema penerimaan mahasiswa baru. Konfigurasi koneksi dan locale berada di file .env. Locale saya set ke id agar teks seperti keterangan waktu tampil dalam Bahasa Indonesia, sedangkan koneksi saya arahkan ke MySQL pada XAMPP, bukan SQLite bawaan.

C.1 Menyiapkan Project dan Memasang HMVC

Project dibuat dengan Laravel Installer memakai starter kit React, autentikasi bawaan, dan Pest sebagai framework pengujian. Setelah project terbentuk, saya memasang paket HMVC

dan mengizinkan plugin merge Composer. Blok merge-plugin saya tambahkan ke bagian extra pada composer.json agar Composer ikut membaca composer.json milik setiap modul, sehingga kelas di dalam folder Modules ter-autoload tanpa konfigurasi PSR-4 manual. Sesudah mengubah composer.json saya menjalankan composer dump-autoload.

Dua modul saya buat sekaligus dengan php artisan module:make UserManagement Pendaftaran. Karena tampilan modul ditulis dengan React, folder resources/views, resources/assets, vite.config.js, dan package.json bawaan modul saya hapus, lalu saya buat folder resources/js/pages di dalam tiap modul. Route API bawaan modul yang memakai middleware auth:sanctum juga saya kosongkan karena Sanctum tidak dipasang pada project ini.

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use App\Http\Middleware\HandleAppearance;
use App\Http\Middleware\HandleInertiaRequests;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Middleware\AddLinkHeadersForPreloadedAssets;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

        $middleware->web(append: [
            HandleAppearance::class,
            HandleInertiaRequests::class,
            AddLinkHeadersForPreloadedAssets::class,
        ]);

        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

Kode 1. Alias middleware role pada bootstrap/app.php.

C.2 Mengatur Halaman React di Dalam Modul

Agar Inertia dapat menemukan halaman milik modul, bagian resolve pada createInertiaApp saya ubah. Vite mengumpulkan seluruh halaman React dari resources/js/pages dan Modules/*/resources/js/pages. Jika nama halaman memuat tanda ::, misalnya Pendaftaran::index, nama itu dipecah menjadi nama modul dan nama halaman, lalu file Modules/Pendaftaran/resources/js/pages/index.tsx dimuat. Tanpa tanda tersebut, halaman dicari di resources/js/pages seperti biasa.

```tsx
import '../css/app.css';

import { createInertiaApp } from '@inertiajs/react';
import { resolvePageComponent } from 'laravel-vite-plugin/inertia-helpers';
import { createRoot } from 'react-dom/client';
import { route as routeFn } from 'ziggy-js';
import { initializeTheme } from './hooks/use-appearance';
import { StrictMode } from 'react';

declare global {
    const route: typeof routeFn;
}

const appName = import.meta.env.VITE_APP_NAME || 'Laravel';

createInertiaApp({
    title: (title) => (title ? `${title} - ${appName}` : appName),
    resolve: (name) => {
        const pages = import.meta.glob([
            './pages/**/*.tsx',
            '../../Modules/*/resources/js/pages/**/*.tsx',
        ]);
        // Halaman modul ditulis dengan format NamaModul::nama-halaman
        if (name.includes('::')) {
            const [module, page] = name.split('::');

            return resolvePageComponent(
                `../../Modules/${module}/resources/js/pages/${page}.tsx`,
                pages,
            );
        }
        return resolvePageComponent(`./pages/${name}.tsx`, pages);
    },
    setup({ el, App, props }) {
        const root = createRoot(el);

        root.render(
            <StrictMode>
                <App {...props} />
            </StrictMode>
        );
    },
    progress: {
        color: '#4B5563',
    }
});

// This will set light / dark mode on load...
initializeTheme();
```

Kode 2. Pengaturan resolve halaman modul pada app.tsx.

Folder modul juga saya daftarkan pada Tailwind CSS lewat baris @source di app.css agar class Tailwind yang ditulis di halaman modul ikut dibuat. Tanpa baris ini, halaman modul

tampil tanpa style. Pada tsconfig.json saya menambahkan pola Modules/*/resources/js/**/*.ts dan *.tsx ke bagian include supaya Visual Studio Code mengenali TypeScript di dalam modul.

C.3 Menambahkan Role pada Tabel Users

Kolom role saya tambahkan lewat migration baru supaya tabel users bawaan Laravel tidak diubah langsung. Nilai bawaan kolom ini adalah user, jadi akun yang tidak menyebut role otomatis menjadi user biasa. Nilai role saya bungkus dalam backed enum UserRole dengan dua case, Admin dan User, beserta method label() untuk menampilkan nama role yang ramah dibaca.

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

Kode 3. Enum UserRole.

Model User saya lengkapi dengan cast role ke enum, nilai bawaan role user, serta dua method bantu isAdmin() dan hasRole(). Method isAdmin() dipakai controller dan Form Request, sedangkan hasRole() dipakai middleware role.

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Fortify\TwoFactorAuthenticatable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable, TwoFactorAuthenticatable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'role',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'two_factor_secret',
        'two_factor_recovery_codes',
        'remember_token',
    ];

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'two_factor_confirmed_at' => 'datetime',
            'role' => UserRole::class,
        ];
    }
```

Kode 4. Model User dengan cast role dan method bantu (bagian 1).

```php

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role :
                UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}
```

Kode 5. Model User dengan method hasRole (bagian 2).

Factory diberi state admin() supaya pembuatan akun admin di seeder dan test cukup memanggil User::factory()->admin()->create(). Seeder membuat satu akun admin (admin@example.com) dan satu akun user (user@example.com), keduanya dengan password password.

```php
<?php

namespace Database\Factories;

use App\Enums\UserRole;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;

/**
 * @extends Factory<User>
 */
class UserFactory extends Factory
{
    /**
     * The current password being used by the factory.
     */
    protected static ?string $password;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'role' => UserRole::User,
            'email_verified_at' => now(),
            'password' => static::$password ??= Hash::make('password'),
            'remember_token' => Str::random(10),
            'two_factor_secret' => null,
            'two_factor_recovery_codes' => null,
            'two_factor_confirmed_at' => null,
            // ... kolom bawaan starter kit lainnya (jika ada) dibiarkan
        ];
    }

    public function admin(): static
    {
        return $this->state(fn (array $attributes) => [
            'role' => UserRole::Admin,
        ]);
    }

    /**
     * Indicate that the model's email address should be unverified.
     */
    public function unverified(): static
    {
        return $this->state(fn (array $attributes) => [
            'email_verified_at' => null,
        ]);
    }
}
```

Kode 6. UserFactory dengan state admin().

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        // User::factory(10)->create();

        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

Kode 7. DatabaseSeeder dengan akun admin dan user.

Proses register bawaan Fortify saya sesuaikan lewat CreateNewUser. Akun hasil register selalu diberi role user, dan field role yang dikirim pengguna diabaikan, sehingga pengguna tidak dapat mendaftarkan dirinya sebagai admin. Akun admin hanya dibuat lewat seeder atau oleh admin di modul UserManagement.

C.4 Authentication dan Shared Props

React Starter Kit sudah menyediakan halaman login, register, dan logout lewat Fortify, jadi bagian ini tidak saya tulis ulang. Pesan error login saya terjemahkan ke Bahasa Indonesia melalui file lang/id/auth.php. Untuk menampilkan pesan berhasil dan error dari redirect controller, saya menambahkan key flash pada method share() di HandleInertiaRequests.

```php
<?php

namespace App\Actions\Fortify;

use App\Enums\UserRole;
use App\Models\User;
use Illuminate\Support\Facades\Validator;
use Illuminate\Validation\Rules\Password;
use Laravel\Fortify\Contracts\CreatesNewUsers;

class CreateNewUser implements CreatesNewUsers
{
    /**
     * Validate and create a newly registered user.
     * Self-registered accounts always get the "user" role.
     *
     * @param  array<string, string>  $input
     */
    public function create(array $input): User
    {
        Validator::make($input, [
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                'unique:users,email',
            ],
            'password' => ['required', 'string', 'confirmed', Password::defaults()],
        ])->validate();

        return User::create([
            'name' => $input['name'],
            'email' => $input['email'],
            'password' => $input['password'],
            'role' => UserRole::User,
        ]);
    }
}
```

Kode 8. CreateNewUser yang memaksa role user.

```php
<?php

namespace App\Http\Middleware;

use Illuminate\Http\Request;
use Inertia\Middleware;

class HandleInertiaRequests extends Middleware
{
    /**
     * The root template that's loaded on the first page visit.
     *
     * @see https://inertiajs.com/server-side-setup#root-template
     *
     * @var string
     */
    protected $rootView = 'app';

    /**
     * Determines the current asset version.
     *
     * @see https://inertiajs.com/asset-versioning
     */
    public function version(Request $request): ?string
    {
        return parent::version($request);
    }

    /**
     * Define the props that are shared by default.
     *
     * @see https://inertiajs.com/shared-data
     *
     * @return array<string, mixed>
     */
    public function share(Request $request): array
    {
        return [
            ...parent::share($request),
            'name' => config('app.name'),
            'auth' => [
                'user' => $request->user(),
            ],
            'flash' => [
                'success' => fn () => $request->session()->get('success'),
                'error' => fn () => $request->session()->get('error'),
            ],
            'sidebarOpen' => ! $request->hasCookie('sidebar_state') || $request->cookie('sidebar_state') === 'true',
            // ... key lain bawaan starter kit (misalnya sidebarOpen) dibiarkan
        ];
    }
}
```

Kode 9. Shared props auth dan flash pada HandleInertiaRequests.

Agar shared props mudah dibaca, saya membuat hook useSharedProps yang mengembalikan user, isAdmin, dan flash. Nilai isAdmin dipakai untuk menampilkan menu Kelola User, tetapi

perlindungan sebenarnya tetap dilakukan oleh middleware role di sisi server. Saya juga membuat komponen AlertMessage dan FlashMessage untuk pesan, serta ConfirmDialog untuk dialog konfirmasi hapus yang dipakai bersama oleh kedua modul.

```tsx
import { usePage } from '@inertiajs/react';
export type UserRole = 'admin' | 'user';
export type AuthUser = {
    id: number;
    name: string;
    email: string;
    role: UserRole;
};
type SharedProps = {
    auth: { user: AuthUser };
    flash: { success?: string | null; error?: string | null };
};
export function useSharedProps() {
    const { auth, flash } = usePage<SharedProps>().props;
    return {
        user: auth.user,
        isAdmin: auth.user?.role === 'admin',
        flash,
    };
}
```

Kode 10. Hook useSharedProps.

```tsx
import { CircleAlert, CircleCheck, X } from 'lucide-react';
import { useEffect, useState } from 'react';
import { useSharedProps } from '@/hooks/use-shared-props';
type AlertMessageProps = {
    type: 'success' | 'error';
    message?: string | null;
    onClose?: () => void;
};
export function AlertMessage({ type, message, onClose }: AlertMessageProps) {
    // Pesan berhasil otomatis tertutup setelah 4 detik
    useEffect(() => {
        if (!message || type !== 'success' || !onClose) {
            return;
        }
        const timer = setTimeout(onClose, 4000);
        return () => clearTimeout(timer);
    }, [message, type]);
    if (!message) {
        return null;
    }
    const Icon = type === 'success' ? CircleCheck : CircleAlert;
    const color =
        type === 'success'
    ? 'border-green-200 bg-green-50 text-green-800 dark:border-green-900 dark:bg-green-950 dark:text-green-200'
    : 'border-red-200 bg-red-50 text-red-800 dark:border-red-900 dark:bg-red-950 dark:text-red-200';

    return (
        <div role="alert" className={`flex items-start gap-3 rounded-lg border px-4 py-3 text-sm ${color}`}>
            <Icon className="mt-0.5 size-4 shrink-0" />
            <p className="flex-1">{message}</p>
            {onClose && (
                <button type="button" onClick={onClose} className="opacity-70 hover:opacity-100" aria-label="Tutup">
                    <X className="size-4" />
                </button>
            )}
        </div>
    );
}
export function FlashMessage() {
    const { flash } = useSharedProps();
    const [success, setSuccess] = useState(flash.success);
    const [error, setError] = useState(flash.error);
    // Tampilkan ulang pesan setiap kali server mengirim flash baru
    useEffect(() => {
        setSuccess(flash.success);
        setError(flash.error);
    }, [flash]);
    return (
        <>
            <AlertMessage type="success" message={success} onClose={() => setSuccess(null)} />
            <AlertMessage type="error" message={error} onClose={() => setError(null)} />
        </>
    );
}
```

Kode 11. Komponen AlertMessage dan FlashMessage.

```tsx
import { Button } from '@/components/ui/button';
import {
    Dialog,
    DialogClose,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog';
type ConfirmDialogProps = {
    open: boolean;
    title?: string;
    description: string;
    confirmLabel?: string;
    processing?: boolean;
    onConfirm: () => void;
    onOpenChange: (open: boolean) => void;
};
export function ConfirmDialog({
    open,
    title = 'Konfirmasi Hapus',
    description,
    confirmLabel = 'Ya, hapus',
    processing = false,
    onConfirm,
    onOpenChange,
}: ConfirmDialogProps) {
    return (
        <Dialog open={open} onOpenChange={onOpenChange}>
            <DialogContent>
                <DialogHeader>
                    <DialogTitle>{title}</DialogTitle>

                    <DialogDescription>{description}</DialogDescription>

                </DialogHeader>
                <DialogFooter className="gap-2">
                    <DialogClose asChild>
                        <Button variant="secondary" disabled={processing}>
                            Batal
                        </Button>
                    </DialogClose>
                    <Button variant="destructive" onClick={onConfirm}
                        disabled={processing}>
                        {processing ? 'Menghapus...' : confirmLabel}
                    </Button>
                </DialogFooter>
            </DialogContent>
        </Dialog>
    );
}
```

Kode 12. Komponen ConfirmDialog.

Menu sidebar saya susun pada app-sidebar.tsx. Menu Kelola User hanya ditambahkan jika isAdmin bernilai true, sedangkan menu Dashboard dan Pendaftaran selalu tampil. NavMain bawaan otomatis memberi tanda aktif pada menu yang sedang dibuka.

```tsx
import { Link } from "@inertiajs/react";
import { ClipboardList, LayoutGrid, Users } from "lucide-react";
import AppLogo from "@/components/app-logo";
import {NavMain} from "@/components/nav-main";
import {NavUser} from "@/components/nav-user";
import {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarHeader,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
} from "@/components/ui/sidebar";
import { useSharedProps } from "@/hooks/use-shared-props";
import type { NavItem } from "@/types";

export function AppSidebar() {
  const { isAdmin } = useSharedProps();

  const mainNavItems: NavItem[] = [
    { title: 'Dashboard', url: '/dashboard', icon: LayoutGrid },
    { title: 'Pendaftaran', url: '/pendaftaran', icon: ClipboardList },
    ...(isAdmin ? [{ title: 'Kelola User', url: '/admin/users', icon: Users }] : []),
  ];

  return (
    <Sidebar collapsible="icon" variant="inset">
      <SidebarHeader>
        <SidebarMenu>
          <SidebarMenuItem>
            <SidebarMenuButton size="lg" asChild>
              <Link href="/dashboard" prefetch>
                <AppLogo />
              </Link>
            </SidebarMenuButton>
          </SidebarMenuItem>
        </SidebarMenu>
      </SidebarHeader>

      <SidebarContent>
        <NavMain items={mainNavItems} />
      </SidebarContent>

      <SidebarFooter>
        <NavUser />
      </SidebarFooter>
    </Sidebar>
  );
}
```

Kode 13. Menu sidebar berbasis role pada app-sidebar.tsx.

C.5 Dashboard

Halaman dashboard dirender oleh DashboardController yang bersifat invokable. Controller mengirim total pendaftar dari model Daftar milik modul Pendaftaran, sedangkan total user hanya dihitung untuk admin. Halaman dashboard.tsx menerima kedua props itu dan menampilkan kartu sesuai role pengguna.

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
    /**
     * Show the dashboard with record counts per module.
     */
    public function __invoke(Request $request): Response
    {
        return Inertia::render('dashboard', [
            'totalPendaftar' => Daftar::count(),
            'totalUsers' => $request->user()->isAdmin() ? User::count() : null,
        ]);
    }
}
```

Kode 14. DashboardController.

```php
<?php

use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;
use Laravel\Fortify\Features;

Route::get('/', function () {
    return Inertia::render('welcome', [
        'canRegister' => Features::enabled(Features::registration()),
    ]);
})->name('home');
Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('dashboard', DashboardController::class)->name('dashboard');
});

require __DIR__.'/settings.php';
```

Kode 15. Route dashboard pada routes/web.php.

```tsx
import { Head, Link } from '@inertiajs/react';
import { ClipboardList, ShieldCheck, Users } from 'lucide-react';
import { useSharedProps } from '@/hooks/use-shared-props';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';

type DashboardProps = {
    totalPendaftar: number;
    totalUsers: number | null;
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Dashboard', href: '/dashboard' }];

export default function Dashboard({ totalPendaftar, totalUsers }: DashboardProps) {
    const { user, isAdmin } = useSharedProps();

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Dashboard" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div className="rounded-xl border border-sidebar-border/70 p-6 dark:border-sidebar-border">
                    <h1 className="text-xl font-semibold">Selamat datang, {user.name}!</h1>
                    <p className="mt-1 flex items-center gap-2 text-sm text-muted-foreground">
                        <ShieldCheck className="size-4" />
                        Anda login sebagai {isAdmin ? 'Admin' : 'User'}.
                    </p>
                </div>

                <div className="grid gap-4 md:grid-cols-2">
                    <Link
                        href="/pendaftaran"
                        className="rounded-xl border border-sidebar-border/70 p-6 transition hover:bg-accent dark:border-sidebar-border"
                    >
                        <ClipboardList className="size-6 text-muted-foreground" />
                        <p className="mt-4 text-3xl font-bold">{totalPendaftar}</p>
                        <p className="text-sm text-muted-foreground">Jumlah pendaftar</p>
                    </Link>

                    {isAdmin && totalUsers !== null && (
                        <Link
                            href="/admin/users"
                            className="rounded-xl border border-sidebar-border/70 p-6 transition hover:bg-accent dark:border-sidebar-border"
                        >
                            <Users className="size-6 text-muted-foreground" />
                            <p className="mt-4 text-3xl font-bold">{totalUsers}</p>
                            <p className="text-sm text-muted-foreground">Jumlah user</p>
                        </Link>
                    )}
                </div>
            </div>
        </AppLayout>
    );
}
```

Kode 16. Halaman dashboard.tsx.

C.6 Authorization dengan Middleware Role

Middleware EnsureUserHasRole menerima daftar role sebagai parameter, misalnya role:admin atau role:admin,user. Jika pengguna belum login atau role-nya tidak termasuk daftar, permintaan dihentikan dengan abort(403) beserta pesan berbahasa Indonesia. Middleware ini saya daftarkan dengan alias role pada bootstrap/app.php, karena Laravel versi ini tidak lagi memakai app/Http/Kernel.php.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Handle an incoming request.
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();
        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

Kode 17. Middleware EnsureUserHasRole.

C.7 Modul UserManagement (CRUD Inertia)

Modul UserManagement memakai model App\Models\User milik aplikasi utama, jadi modul ini tidak memiliki model sendiri. Aturan validasi saya taruh pada StoreUserRequest dan UpdateUserRequest. Pada UpdateUserRequest, email memakai ignore agar email milik user yang sedang diedit tidak dianggap duplikat, dan password bersifat nullable sehingga password lama tetap dipakai bila kolomnya dikosongkan. Rule::when membatasi pilihan role ketika admin mengubah akunnya sendiri.

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => ['required', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

Kode 18. StoreUserRequest.

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', Rule::unique('users', 'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

Kode 19. UpdateUserRequest dengan aturan role untuk akun sendiri.

Controller mendukung pencarian nama atau email, filter role, dan pagination sepuluh baris per halaman. Setiap aksi mengembalikan halaman Inertia atau redirect beserta pesan flash. Method destroy menolak penghapusan akun sendiri dengan pesan error.

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): Response
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString()
            ->through(fn (User $user) => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
                'role_label' => $user->role->label(),
                'created_at' => $user->created_at?->diffForHumans(),
            ]);

        return Inertia::render('UserManagement::index', [
            'users' => $users,
            'filters' => [
                'search' => $search,
                'role' => $role?->value ?? '',
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): Response
    {
        return Inertia::render('UserManagement::create', [
            'roles' => $this->roleOptions(),
        ]);
    }
```

Kode 20. UserManagementController bagian 1 (index).

```php

    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')->with('success', 'User berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified user.
     */
    public function edit(User $user): Response
    {
        return Inertia::render('UserManagement::edit', [
            'user' => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Update the specified user. The password is only changed when filled in.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);

        return redirect()->route('admin.users.index')->with('success', 'User berhasil diperbarui.');
    }

    /**
     * Remove the specified user. An admin cannot delete their own account.
     */
    public function destroy(Request $request, User $user): RedirectResponse
    {
        if ($user->is($request->user())) {
            return redirect()->route('admin.users.index')->with('error', 'Anda tidak dapat menghapus akun Anda sendiri.');
        }

        $user->delete();

        return redirect()->route('admin.users.index')->with('success', 'User berhasil dihapus.');
    }
```

Kode 21. UserManagementController bagian 2 (store, edit, update).

```php

    /**
     * Role options for the select input on the React page.
     *
     * @return list<array{value: string, label: string}>
     */
    private function roleOptions(): array
    {
        return array_map(
            fn (UserRole $role) => ['value' => $role->value, 'label' => $role->label()],
            UserRole::cases()
        );
    }
}
```

Kode 22. UserManagementController bagian 3 (destroy, roleOptions).

Route modul didaftarkan dengan Route::resource di dalam grup middleware auth dan role:admin, memakai prefix admin dan awalan nama admin., sehingga URL-nya menjadi /admin/users.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)->except('show');
    });
```

Kode 23. Route modul UserManagement.

Halaman index.tsx menampilkan tabel user, kotak pencarian, filter role, dan pagination. Tombol Hapus tidak ditampilkan pada baris akun yang sedang login, dan penghapusan memakai ConfirmDialog sehingga request DELETE baru dikirim setelah pengguna menekan Ya, hapus. Halaman create dan edit memakai satu komponen form bersama, user-form.tsx, dengan useForm milik Inertia.

```tsx
import { Head, Link, router } from '@inertiajs/react';
import { Pencil, Plus, Search, Trash2 } from 'lucide-react';
import { useState } from 'react';
import type { FormEvent } from 'react';
import { FlashMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { useSharedProps } from '@/hooks/use-shared-props';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';

type UserRow = {
    id: number;
    name: string;
    email: string;
    role: 'admin' | 'user';
    role_label: string;
    created_at: string | null;
};

type PaginationLink = { url: string | null; label: string; active: boolean };

type IndexProps = {
    users: {
        data: UserRow[];
        links: PaginationLink[];
        from: number | null;
        to: number | null;
        total: number;
    };
    filters: { search: string; role: string };
    roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Kelola User', href: '/admin/users' }];

export default function Index({ users, filters, roles }: IndexProps) {
    const { user: currentUser } = useSharedProps();
    const [search, setSearch] = useState(filters.search);
    const [deleting, setDeleting] = useState<UserRow | null>(null);
    const [processing, setProcessing] = useState(false);

    // Kirim ulang request GET /admin/users dengan kata kunci dan filter role
    const applyFilters = (params: { search?: string; role?: string }) => {
        router.get('/admin/users', { ...filters, search, ...params }, { preserveState:
true, replace: true });
    };

    const handleSearch = (event: FormEvent) => {
        event.preventDefault();
        applyFilters({ search });
    };

    const handleDelete = () => {
        if (!deleting) {
            return;
        }
        router.delete(`/admin/users/${deleting.id}`, {
            preserveScroll: true,
            onStart: () => setProcessing(true),
            onFinish: () => {
                setProcessing(false);
                setDeleting(null);
            },
        });
    };
```

Kode 24. index.tsx modul UserManagement bagian 1 (tipe dan logika filter).

```tsx

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Kelola User" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div className="flex flex-wrap items-center justify-between gap-2">
                    <div>
                        <h1 className="text-xl font-semibold">Kelola User</h1>
                        <p className="text-sm text-muted-foreground">Total {users.total} user terdaftar.</p>
                    </div>
                    <Button asChild>
                        <Link href="/admin/users/create">
                            <Plus /> Tambah User
                        </Link>
                    </Button>
                </div>

                <FlashMessage />

                <div className="flex flex-wrap gap-2">
                    <form onSubmit={handleSearch} className="flex flex-1 gap-2">
                        <Input
                            value={search}
                            onChange={(event) => setSearch(event.target.value)}
                            placeholder="Cari nama atau email..."
                            className="max-w-sm"
                        />
                        <Button type="submit" variant="secondary">
                            <Search /> Cari
                        </Button>
                    </form>
                    <select
                        value={filters.role}
                        onChange={(event) => applyFilters({ role: event.target.value })}
                        className="h-9 rounded-md border border-input bg-transparent px-3 text-sm shadow-xs"
                    >
                        <option value="">Semua role</option>
                        {roles.map((role) => (
                            <option key={role.value} value={role.value}>
                                {role.label}
                            </option>
                        ))}
                    </select>
                </div>
```

Kode 25. index.tsx modul UserManagement bagian 2 (markup tabel).

```tsx

                <div className="overflow-x-auto rounded-xl border">
                    <table className="w-full text-sm">
                        <thead className="bg-muted/50 text-left">
                            <tr>
                                <th className="px-4 py-3 font-medium">No</th>
                                <th className="px-4 py-3 font-medium">Nama</th>
                                <th className="px-4 py-3 font-medium">Email</th>
                                <th className="px-4 py-3 font-medium">Role</th>
                                <th className="px-4 py-3 font-medium">Dibuat</th>
                                <th className="px-4 py-3 text-right font-medium">Aksi</th>

                            </tr>
                        </thead>
                        <tbody>
                            {users.data.length === 0 && (
                                <tr>
                                    <td colSpan={6} className="px-4 py-8 text-center text-
muted-foreground">
                                        Data user tidak ditemukan.
                                    </td>
                                </tr>
                            )}
                            {users.data.map((user, index) => (
                                <tr key={user.id} className="border-t">
                                    <td className="px-4 py-3">{(users.from ?? 1) + index}
</td>
                                    <td className="px-4 py-3 font-medium">
                                        {user.name}
                                        {user.id === currentUser.id && (
                                            <Badge variant="outline" className="ml-2">
                                                Anda
                                            </Badge>
                                        )}
                                    </td>
                                    <td className="px-4 py-3">{user.email}</td>
                                    <td className="px-4 py-3">
                                        <Badge variant={user.role === 'admin' ? 'default'
: 'secondary'}>{user.role_label}</Badge>
                                    </td>
                                    <td className="px-4 py-3 text-muted-foreground">
{user.created_at}</td>
                                    <td className="px-4 py-3">
                                        <div className="flex justify-end gap-2">
                                            <Button asChild size="sm" variant="outline">
                                                <Link href=
{`/admin/users/${user.id}/edit`}>
                                                    <Pencil /> Edit
                                                </Link>
                                            </Button>
                                            {user.id !== currentUser.id && (
                                                <Button size="sm" variant="destructive"
onClick={() => setDeleting(user)}>
                                                    <Trash2 /> Hapus
                                                </Button>
                                            )}
                                        </div>
                                    </td>
                                </tr>
                            ))}
                        </tbody>
                    </table>
                </div>
```

Kode 26. index.tsx modul UserManagement bagian 3 (baris tabel dan pagination).

```
                <div className="flex flex-wrap items-center justify-between gap-2 text-sm">
                    <p className="text-muted-foreground">
                        Menampilkan {users.from ?? 0}-{users.to ?? 0} dari {users.total} data
                    </p>
                    <div className="flex flex-wrap gap-1">
                        {users.links.map((link, index) =>
                            link.url ? (
                                <Button key={index} asChild size="sm" variant={link.active ? 'default' : 'outline'}>
                                    <Link href={link.url} preserveScroll>
                                        {cleanLabel(link.label)}
                                    </Link>
                                </Button>
                            ) : (
                                <Button key={index} size="sm" variant="outline" disabled>
                                    {cleanLabel(link.label)}
                                </Button>
                            ),
                        )}
                    </div>
                </div>
            </div>

            <ConfirmDialog
                open={deleting !== null}
                description={`Yakin ingin menghapus user ${deleting?.name}? Tindakan ini tidak dapat dibatalkan.`}
                processing={processing}
                onConfirm={handleDelete}
                onOpenChange={(open) => !open && setDeleting(null)}
            />
        </AppLayout>
    );
}

// Label pagination Laravel berisi entitas HTML &laquo; dan &raquo;
function cleanLabel(label: string): string {
    return label.replace('&laquo;', '«').replace('&raquo;', '»');
}
```

Kode 27. index.tsx modul UserManagement bagian 4 (ConfirmDialog).

```tsx
import { Link } from '@inertiajs/react';
import type { FormEvent } from 'react';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export type UserFormData = {
    name: string;
    email: string;
    role: string;
    password: string;
    password_confirmation: string;
};

type UserFormProps = {
    data: UserFormData;
    errors: Partial<Record<keyof UserFormData, string>>;
    processing: boolean;
    roles: { value: string; label: string }[];
    isEdit?: boolean;
    submitLabel: string;
    setData: (key: keyof UserFormData, value: string) => void;
    onSubmit: (event: FormEvent) => void;
};

export function UserForm({ data, errors, processing, roles, isEdit = false, submitLabel, setData, onSubmit }: UserFormProps) {
    return (
        <form onSubmit={onSubmit} noValidate className="max-w-xl space-y-5 rounded-xl border p-6">
            <div className="grid gap-2">
                <Label htmlFor="name">
                    Nama <span className="text-red-500">*</span>
                </Label>
                <Input id="name" value={data.name} onChange={(event) => setData('name', event.target.value)} autoFocus />
                <InputError message={errors.name} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="email">
                    Email <span className="text-red-500">*</span>
                </Label>
                <Input id="email" type="email" value={data.email} onChange={(event) => setData('email', event.target.value)} />
                <InputError message={errors.email} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="role">
                    Role <span className="text-red-500">*</span>
                </Label>
                <select
                    id="role"
                    value={data.role}
                    onChange={(event) => setData('role', event.target.value)}
                    className="h-9 rounded-md border border-input bg-transparent px-3 text-sm shadow-xs"
                >
                    {roles.map((role) => (
                        <option key={role.value} value={role.value}>
                            {role.label}
                        </option>
                    ))}
                </select>
                <InputError message={errors.role} />
            </div>
```

Kode 28. Komponen user-form.tsx bagian 1 (input nama, email, role).

```tsx

            <div className="grid gap-2">
                <Label htmlFor="password">
                    Password {!isEdit && <span className="text-red-500">*</span>}
                </Label>
                <Input
                    id="password"
                    type="password"
                    value={data.password}
                    onChange={(event) => setData('password', event.target.value)}
                />
                {isEdit && <p className="text-xs text-muted-foreground">Kosongkan jika tidak ingin mengganti password.</p>}
                <InputError message={errors.password} />
            </div>

            <div className="grid gap-2">
                <Label htmlFor="password_confirmation">Konfirmasi Password</Label>
                <Input
                    id="password_confirmation"
                    type="password"
                    value={data.password_confirmation}
                    onChange={(event) => setData('password_confirmation', event.target.value)}
                />
            </div>

            <div className="flex gap-2">
                <Button type="submit" disabled={processing}>
                    {processing ? 'Menyimpan...' : submitLabel}
                </Button>
                <Button asChild variant="secondary">
                    <Link href="/admin/users">Batal</Link>
                </Button>
            </div>
        </form>
    );
}
```

Kode 29. Komponen user-form.tsx bagian 2 (password dan tombol).

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';

type CreateProps = {
    roles: { value: string; label: string }[];
};

const breadcrumbs: BreadcrumbItem[] = [
    { title: 'Kelola User', href: '/admin/users' },
    { title: 'Tambah User', href: '/admin/users/create' },
];

export default function Create({ roles }: CreateProps) {
    const { data, setData, post, processing, errors } = useForm<UserFormData>({
        name: '',
        email: '',
        role: 'user',
        password: '',
        password_confirmation: '',
    });

    const submit = (event: FormEvent) => {
        event.preventDefault();
        post('/admin/users');
    };

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Tambah User" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div>
                    <h1 className="text-xl font-semibold">Tambah User</h1>
                    <p className="text-sm text-muted-foreground">Isi data akun baru
beserta role-nya.</p>
                </div>

                <UserForm
                    data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    submitLabel="Simpan"
                    setData={setData}
                    onSubmit={submit}
                />
            </div>
        </AppLayout>
    );
}
```

Kode 30. create.tsx modul UserManagement.

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';

type EditProps = {
    user: { id: number; name: string; email: string; role: string };
    roles: { value: string; label: string }[];
};

export default function Edit({ user, roles }: EditProps) {
    const breadcrumbs: BreadcrumbItem[] = [
        { title: 'Kelola User', href: '/admin/users' },
        { title: 'Edit User', href: `/admin/users/${user.id}/edit` },
    ];

    const { data, setData, put, processing, errors } = useForm<UserFormData>({
        name: user.name,
        email: user.email,
        role: user.role,
        password: '',
        password_confirmation: '',
    });

    const submit = (event: FormEvent) => {
        event.preventDefault();
        put(`/admin/users/${user.id}`);
    };

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Edit User" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div>
                    <h1 className="text-xl font-semibold">Edit User</h1>
                    <p className="text-sm text-muted-foreground">Perbarui data akun
{user.name}.</p>
                </div>

                <UserForm
                    data={data}
                    errors={errors}
                    processing={processing}
                    roles={roles}
                    isEdit
                    submitLabel="Simpan Perubahan"
                    setData={setData}
                    onSubmit={submit}
                />
            </div>
        </AppLayout>
    );
}
```

Kode 31. edit.tsx modul UserManagement.

C.8 Modul Pendaftaran (CRUD AJAX)

Modul Pendaftaran menyimpan data calon mahasiswa beserta satu berkas pendukung. Migration pertama membuat tabel daftars, sedangkan migration kedua menambahkan kolom berkas dan nama_berkas. Pemisahan ini saya lakukan karena kolom berkas ditambahkan setelah tabel awal terbentuk. Kolom no_hp dibuat sepanjang 20 karakter agar angka 0 di depan tidak hilang.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('daftars', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('asal_sekolah');
            $table->string('prodi_tujuan');
            $table->string('email')->unique();
            $table->string('no_hp', 20);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('daftars');
    }
};
```

Kode 32. Migration tabel daftars.



```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->string('berkas')->nullable()->after('no_hp');
            $table->string('nama_berkas')->nullable()->after('berkas');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->dropColumn(['berkas', 'nama_berkas']);
        });
    }
};
```

Kode 33. Migration penambah kolom berkas dan nama_berkas.

Factory dipakai untuk data uji. Model Daftar mengisi $fillable dan menambahkan accessor berkas_url yang diappend ke JSON. Accessor itu mengubah path penyimpanan menjadi URL lengkap lewat asset('storage/...'), sehingga React tidak perlu menyusun alamat berkas sendiri. Factory milik modul didaftarkan melalui newFactory() karena letaknya di luar folder factories bawaan Laravel.

```php
<?php

namespace Modules\Pendaftaran\Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use Modules\Pendaftaran\Models\Daftar;

/**
 * @extends Factory<Daftar>
 */
class DaftarFactory extends Factory
{
    /**
     * The name of the factory's corresponding model.
     */
    protected $model = Daftar::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'nama' => fake()->name(),
            'asal_sekolah' => 'SMA '.fake()->city(),
            'prodi_tujuan' => fake()->randomElement(['Informatika', 'Sistem Informasi', 'Teknik Elektro']),
            'email' => fake()->unique()->safeEmail(),
            'no_hp' => fake()->numerify('08##########'),
        ];
    }
}
```

Kode 34. DaftarFactory untuk data uji.

```php
<?php

namespace Modules\Pendaftaran\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
    /** @use HasFactory<DaftarFactory> */
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];

    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ? asset('storage/'.$this->berkas) : null);
    }

    protected static function newFactory(): DaftarFactory
    {
        return DaftarFactory::new();
    }
}
```

Kode 35. Model Daftar dengan accessor berkas_url.

DaftarRequest memakai aturan berkas yang berbeda untuk tambah dan ubah. Saat tambah, berkas wajib diunggah, sedangkan saat edit boleh dikosongkan karena berkas lama tetap

dipakai. Format dibatasi jpg, jpeg, png, atau pdf dengan ukuran maksimal 2 MB, dan pesan galat saya tulis dalam Bahasa Indonesia lewat method messages().

```php
<?php

namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $daftar = $this->route('daftar');

        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255', Rule::unique('daftars', 'email')->ignore($daftar?->id)],
            'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
            'berkas' => [$daftar ? 'nullable' : 'required', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
}
```

Kode 36. DaftarRequest dengan validasi berkas.

Controller hanya memakai satu method untuk memuat halaman, karena data tabel diambil terpisah lewat getData(). Method store, update, dan destroy mengembalikan JSON berisi pesan berhasil. Berkas disimpan dengan store() ke storage/app/public, sedangkan database hanya menyimpan path dan nama asli berkas. Saat berkas diganti atau data dihapus, berkas lama ikut dihapus dari storage.

```php
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    /**
     * Display the React page. Data is loaded via AJAX.
     */
    public function index(): Response
    {
        return Inertia::render('Pendaftaran::index');
    }

    /**
     * Return all registrations as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }

    /**
     * Store a newly created registration together with its uploaded file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');
        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }

    /**
     * Update the specified registration. The old file is replaced only when a new one is
uploaded.
     */
    public function update(DaftarRequest $request, Daftar $daftar): JsonResponse
    {
        $data = $request->safe()->except('berkas');
```

Kode 37. PendaftaranController bagian 1 (index, getData, store).

```php

        if ($request->hasFile('berkas')) {
            if ($daftar->berkas) {
                Storage::disk('public')->delete($daftar->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $daftar->update($data);

        return response()->json(['success' => 'Data berhasil diperbarui.']);
    }

    /**
     * Remove the specified registration and its uploaded file.
     */
    public function destroy(Daftar $daftar): JsonResponse
    {
        if ($daftar->berkas) {
            Storage::disk('public')->delete($daftar->berkas);
        }

        $daftar->delete();

        return response()->json(['success' => 'Data berhasil dihapus.']);
    }
}
```

Kode 38. PendaftaranController bagian 2 (edit, update, destroy).

Route modul dilindungi middleware auth dan role:admin,user. Route pendaftaran/data saya tulis sebelum route pendaftaran/{daftar} agar kata data tidak dianggap sebagai id.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('pendaftaran', [PendaftaranController::class, 'index'])->name('pendaftaran.index');
    Route::get('pendaftaran/data', [PendaftaranController::class, 'getData'])->name('pendaftaran.data');
    Route::post('pendaftaran', [PendaftaranController::class, 'store'])->name('pendaftaran.store');
    Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class, 'edit'])->name('pendaftaran.edit');
    Route::put('pendaftaran/{daftar}', [PendaftaranController::class, 'update'])->name('pendaftaran.update');
    Route::delete('pendaftaran/{daftar}', [PendaftaranController::class, 'destroy'])->name('pendaftaran.destroy');
});
```

Kode 39. Route modul Pendaftaran.

Untuk mengirim request AJAX saya membuat helper http.ts di resources/js/lib. Helper ini membaca token CSRF dari cookie XSRF-TOKEN lalu mengirimnya sebagai header X-XSRF-TOKEN, dan menambahkan Accept: application/json agar Laravel mengembalikan error validasi sebagai JSON berkode 422. Jika respons bernilai gagal, helper melempar HttpError yang membawa status dan isi JSON.

```ts
export type ValidationErrors = Record<string, string[]>;

type ErrorBody = { message?: string; errors?: ValidationErrors } | null;

export class HttpError extends Error {
    status: number;
    data: ErrorBody;

    constructor(status: number, data: ErrorBody) {
        super(data?.message ?? `Request gagal dengan status ${status}`);
        this.status = status;
        this.data = data;
    }
}

// Membaca token CSRF dari cookie XSRF-TOKEN yang dibuat oleh Laravel
function getXsrfToken(): string {
    const cookie = document.cookie.split('; ').find((row) => row.startsWith('XSRF-TOKEN='));

    return cookie ? decodeURIComponent(cookie.substring('XSRF-TOKEN='.length)) : '';
}

type RequestOptions = {
    method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
    body?: FormData;
};

export async function http<T>(url: string, { method = 'GET', body }: RequestOptions = {}): Promise<T> {
    const response = await fetch(url, {
        method,
        body,
        credentials: 'same-origin',
        headers: {
            Accept: 'application/json',
            'X-Requested-With': 'XMLHttpRequest',
            'X-XSRF-TOKEN': getXsrfToken(),
        },
    });

    const data = await response.json().catch(() => null);

    if (!response.ok) {
        throw new HttpError(response.status, data);
    }

    return data as T;
}
```

Kode 40. Helper http.ts untuk request AJAX.

Halaman index.tsx memuat data sekali lewat GET /pendaftaran/data, lalu seluruh operasi tambah, ubah, dan hapus berjalan tanpa memuat ulang halaman. Form tambah dan edit memakai satu Dialog yang sama; state editing bernilai null saat menambah dan berisi data

pendaftar saat edit. Seluruh form dikirim sebagai FormData karena data berisi file, dan saat edit ditambahkan _method PUT karena PHP hanya membaca file pada request POST.

```tsx
import { Head } from '@inertiajs/react';
import { FileImage, FileText, LoaderCircle, Pencil, Plus, Search, Trash2 } from 'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import type { ChangeEvent, FormEvent } from 'react';
import { AlertMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import InputError from '@/components/input-error';
import { Badge } from '@/components/ui/badge';
import { Button } from '@/components/ui/button';
import {
    Dialog,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';
import AppLayout from '@/layouts/app-layout';
import { http, HttpError } from '@/lib/http';
import type { ValidationErrors } from '@/lib/http';
import type { BreadcrumbItem } from '@/types';

type Daftar = {
    id: number;
    nama: string;
    asal_sekolah: string;
    prodi_tujuan: string;
    email: string;
    no_hp: string;
    berkas: string | null;
    nama_berkas: string | null;
    berkas_url: string | null;
};

type FormFields = Pick<Daftar, 'nama' | 'asal_sekolah' | 'prodi_tujuan' | 'email' | 'no_hp'>;
type Message = { type: 'success' | 'error'; text: string } | null;

const emptyForm: FormFields = { nama: '', asal_sekolah: '', prodi_tujuan: '', email: '', no_hp: '' };

const fields: { name: keyof FormFields; label: string; type?: string }[] = [
    { name: 'nama', label: 'Nama' },
    { name: 'asal_sekolah', label: 'Asal Sekolah' },
    { name: 'prodi_tujuan', label: 'Prodi Tujuan' },
    { name: 'email', label: 'Email', type: 'email' },
    { name: 'no_hp', label: 'No HP' },
];

const urls = {
    data: '/pendaftaran/data',
    store: '/pendaftaran',
    edit: (id: number) => `/pendaftaran/${id}/edit`,
    update: (id: number) => `/pendaftaran/${id}`,
    destroy: (id: number) => `/pendaftaran/${id}`,
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Pendaftaran', href: '/pendaftaran' }];
```

Kode 41. index.tsx modul Pendaftaran bagian 1 (tipe dan url).

```tsx

// Tautan berkas yang dibuka di tab baru, ikon dipilih berdasarkan ekstensi
function FileLink({ url, name }: { url: string | null; name: string | null }) {
    if (!url) {
        return <span className="text-muted-foreground">-</span>;
    }

    const Icon = name?.toLowerCase().endsWith('.pdf') ? FileText : FileImage;

    return (
        <a
            href={url}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-1 text-primary underline-offset-4
hover:underline"
        >
            <Icon className="size-4" /> {name ?? 'Lihat berkas'}
        </a>
    );
}

export default function Index() {
    const [records, setRecords] = useState<Daftar[]>([]);
    const [loading, setLoading] = useState(true);
    const [keyword, setKeyword] = useState('');
    const [message, setMessage] = useState<Message>(null);
    const [dialogOpen, setDialogOpen] = useState(false);
    const [editing, setEditing] = useState<Daftar | null>(null);
    const [form, setForm] = useState<FormFields>(emptyForm);
    const [file, setFile] = useState<File | null>(null);
    const [fileInputKey, setFileInputKey] = useState(0);
    const [errors, setErrors] = useState<ValidationErrors>({});
    const [saving, setSaving] = useState(false);
    const [deleting, setDeleting] = useState<Daftar | null>(null);
    const [processingDelete, setProcessingDelete] = useState(false);

    const showError = (error: unknown, fallback: string) => {
        const text = error instanceof HttpError ? (error.data?.message ?? fallback) :
fallback;
        setMessage({ type: 'error', text });
    };

    // Mengambil seluruh data pendaftar dari server (GET /pendaftaran/data)
    const loadData = async () => {
        setLoading(true);
        try {
            setRecords(await http<Daftar[]>(urls.data));
        } catch (error) {
            showError(error, 'Gagal memuat data pendaftar.');
        } finally {
            setLoading(false);
        }
    };

    useEffect(() => {
        loadData();
    }, []);

    // Pencarian dilakukan di browser tanpa request baru ke server
    const filtered = useMemo(() => {
        const q = keyword.trim().toLowerCase();
```

Kode 42. index.tsx modul Pendaftaran bagian 2 (FileLink dan state halaman).

```
        if (q === '') {
            return records;
        }

        return records.filter((item) =>
            [item.nama, item.asal_sekolah, item.prodi_tujuan, item.email,
item.no_hp].some((value) =>
                value.toLowerCase().includes(q),
            ),
        );
    }, [records, keyword]);

    const resetFileAndErrors = () => {
        setFile(null);
        setFileInputKey((key) => key + 1);
        setErrors({});
    };

    const openCreate = () => {
        setEditing(null);
        setForm(emptyForm);
        resetFileAndErrors();
        setDialogOpen(true);
    };

    // Mengambil satu data (GET /pendaftaran/{id}/edit) lalu mengisi form di dialog
    const openEdit = async (id: number) => {
        try {
            const data = await http<Daftar>(urls.edit(id));

            setEditing(data);
            setForm({
                nama: data.nama,
                asal_sekolah: data.asal_sekolah,
                prodi_tujuan: data.prodi_tujuan,
                email: data.email,
                no_hp: data.no_hp,
            });
            resetFileAndErrors();
            setDialogOpen(true);
        } catch (error) {
            showError(error, 'Data tidak ditemukan.');
        }
    };

    const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
        setForm({ ...form, [event.target.name]: event.target.value });
    };

    // Menyimpan data (tambah dan edit) menggunakan FormData
    const handleSubmit = async (event: FormEvent) => {
        event.preventDefault();
        setSaving(true);
        setErrors({});

        const formData = new FormData();

        Object.entries(form).forEach(([key, value]) => formData.append(key, value));

        if (file) {
            formData.append('berkas', file);
        }
```

Kode 43. index.tsx modul Pendaftaran bagian 3 (memuat data dan mencari).

```
        if (editing) {
            formData.append('_method', 'PUT');
        }

        try {
            const result = await http<{ success: string }>(editing ? urls.update(editing.id) : urls.store, {
                method: 'POST',
                body: formData,
            });

            setDialogOpen(false);
            setMessage({ type: 'success', text: result.success });
            await loadData();
        } catch (error) {
            if (error instanceof HttpError && error.status === 422) {
                setErrors(error.data?.errors ?? {});
            } else {
                showError(error, 'Terjadi kesalahan, silakan coba lagi.');
            }
        } finally {
            setSaving(false);
        }
    };

    // Menghapus data (DELETE /pendaftaran/{id}) setelah dikonfirmasi
    const handleDelete = async () => {
        if (!deleting) {
            return;
        }

        setProcessingDelete(true);
        try {
            const result = await http<{ success: string }>(urls.destroy(deleting.id), { method: 'DELETE' });
            setMessage({ type: 'success', text: result.success });
            await loadData();
        } catch (error) {
            showError(error, 'Data gagal dihapus.');
        } finally {
            setProcessingDelete(false);
            setDeleting(null);
        }
    };

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Pendaftaran" />

            <div className="flex flex-1 flex-col gap-4 p-4">
                <div className="flex flex-wrap items-center justify-between gap-2">
                    <div>
                        <h1 className="flex items-center gap-2 text-xl font-semibold">
                            Data Pendaftar <Badge variant="secondary">{records.length}</Badge>
                        </h1>
                        <p className="text-sm text-muted-foreground">
                            Seluruh operasi pada halaman ini berjalan dengan AJAX tanpa memuat ulang halaman.
                        </p>
                    </div>
                    <Button onClick={openCreate}>
                        <Plus /> Tambah Pendaftar
                    </Button>
                </div>
```

Kode 44. index.tsx modul Pendaftaran bagian 4 (simpan data dengan AJAX).

```tsx

                <AlertMessage type={message?.type ?? 'success'} message={message?.text}
onClose={() => setMessage(null)} />

                <div className="relative max-w-sm">
                    <Search className="absolute top-1/2 left-3 size-4 -translate-y-1/2
text-muted-foreground" />
                    <Input
                        value={keyword}
                        onChange={(event) => setKeyword(event.target.value)}
                        placeholder="Cari nama, sekolah, prodi, email..."
                        className="pl-9"
                    />
                </div>

                <div className="overflow-x-auto rounded-xl border">
                    <table className="w-full text-sm">
                        <thead className="bg-muted/50 text-left">
                            <tr>
                                <th className="px-4 py-3 font-medium">No</th>
                                <th className="px-4 py-3 font-medium">Nama</th>
                                <th className="px-4 py-3 font-medium">Asal Sekolah</th>
                                <th className="px-4 py-3 font-medium">Prodi Tujuan</th>
                                <th className="px-4 py-3 font-medium">Email</th>
                                <th className="px-4 py-3 font-medium">No HP</th>
                                <th className="px-4 py-3 font-medium">Berkas</th>
                                <th className="px-4 py-3 text-right font-medium">Aksi</th>

                            </tr>
                        </thead>
                        <tbody>
                            {loading && (
                                <tr>
                                    <td colSpan={8} className="px-4 py-8 text-center text-
muted-foreground">
                                        <LoaderCircle className="mx-auto size-5 animate-
spin" />
                                    </td>
                                </tr>
                            )}
                            {!loading && filtered.length === 0 && (
                                <tr>
                                    <td colSpan={8} className="px-4 py-8 text-center text-
muted-foreground">
                                        {records.length === 0 ? 'Belum ada data
pendaftar.' : 'Data tidak ditemukan.'}
                                    </td>
                                </tr>
                            )}
                            {!loading &&
                                filtered.map((item, index) => (
                                    <tr key={item.id} className="border-t">
                                        <td className="px-4 py-3">{index + 1}</td>
                                        <td className="px-4 py-3 font-medium">{item.nama}
</td>
                                        <td className="px-4 py-3">{item.asal_sekolah}</td>

                                        <td className="px-4 py-3">{item.prodi_tujuan}</td>

                                        <td className="px-4 py-3">{item.email}</td>
                                        <td className="px-4 py-3">{item.no_hp}</td>
                                        <td className="px-4 py-3">
                                            <FileLink url={item.berkas_url} name=
{item.nama_berkas} />
                                        </td>
                                        <td className="px-4 py-3">
                                            <div className="flex justify-end gap-2">
                                                <Button size="sm" variant="outline"
onClick={() => openEdit(item.id)}>
                                                    <Pencil /> Edit
                                                </Button>
                                                <Button size="sm" variant="destructive"
onClick={() => setDeleting(item)}>
                                                    <Trash2 /> Hapus
                                                </Button>
                                            </div>
                                        </td>
                                    </tr>
                                ))}
                        </tbody>
                    </table>
                </div>
            </div>
```

Kode 45. index.tsx modul Pendaftaran bagian 5 (markup tabel).



```
            <Dialog open={dialogOpen} onOpenChange={(open) => !saving &&
setDialogOpen(open)}>
                <DialogContent className="sm:max-w-lg">
                    <form onSubmit={handleSubmit} noValidate className="space-y-4">
                        <DialogHeader>
                            <DialogTitle>{editing ? 'Edit Pendaftar' : 'Tambah Pendaftar'}
</DialogTitle>
                            <DialogDescription>Lengkapi data calon mahasiswa beserta
berkas pendukungnya.</DialogDescription>
                        </DialogHeader>

                        {fields.map((field) => (
                            <div key={field.name} className="grid gap-2">
                                <Label htmlFor={field.name}>
                                    {field.label} <span className="text-red-500">*</span>
                                </Label>
                                <Input
                                    id={field.name}
                                    name={field.name}
                                    type={field.type ?? 'text'}
                                    value={form[field.name]}
                                    onChange={handleChange}
                                    aria-invalid={!!errors[field.name]}
                                />
                                <InputError message={errors[field.name]?.[0]} />
                            </div>
                        ))}

                        <div className="grid gap-2">
                            <Label htmlFor="berkas">
                                Berkas Pendukung {!editing && <span className="text-red-
500">*</span>}
                            </Label>
                            <Input
                                key={fileInputKey}
                                id="berkas"
                                type="file"
                                accept=".jpg,.jpeg,.png,.pdf"
                                onChange={(event) => setFile(event.target.files?.[0] ??
null)}
                                aria-invalid={!!errors.berkas}
                            />
                            <p className="text-xs text-muted-foreground">
                                Format jpg, jpeg, png, atau pdf, maksimal 2 MB.
                                {editing && ' Kosongkan jika tidak ingin mengganti
berkas.'}
                            </p>
                            {editing?.berkas_url && (
                                <p className="text-xs">
                                    Berkas saat ini: <FileLink url={editing.berkas_url}
name={editing.nama_berkas} />
                                </p>
                            )}
                            <InputError message={errors.berkas?.[0]} />
                        </div>

                        <DialogFooter className="gap-2">
                            <Button type="button" variant="secondary" onClick={() =>
setDialogOpen(false)} disabled={saving}>
                                Batal
                            </Button>
                            <Button type="submit" disabled={saving}>
                                {saving && <LoaderCircle className="animate-spin" />}
                                {saving ? 'Menyimpan...' : 'Simpan'}
                            </Button>
                        </DialogFooter>
                    </form>
                </DialogContent>
            </Dialog>
```

Kode 46. index.tsx modul Pendaftaran bagian 6 (dialog form).

```
1
2               <ConfirmDialog
3                   open={deleting !== null}
4                   description={`Yakin ingin menghapus data ${deleting?.nama} beserta
    berkasnya?`}
5                   processing={processingDelete}
6                   onConfirm={handleDelete}
7                   onOpenChange={(open) => !open && setDeleting(null)}
8               />
9           </AppLayout>
10      );
11  }
```

Kode 47. index.tsx modul Pendaftaran bagian 7 (dialog lanjutan dan konfirmasi).

C.9 Pengujian Otomatis

Selain diuji melalui browser, seluruh fitur diuji secara otomatis memakai Pest. React Starter Kit sudah menyediakan pengujian authentication bawaan, jadi saya cukup menambahkan pengujian untuk role dan kedua modul. Pengujian memakai RefreshDatabase sehingga setiap test berjalan pada database kosong. Pengujian modul memastikan tamu dialihkan ke login, akun hasil register selalu berperan user, user tidak dapat membuka halaman admin, admin dapat melihat daftar user dan menambah user, serta admin tidak dapat menurunkan role atau menghapus akunnya sendiri.

```php
<?php

use App\Models\User;
use Inertia\Testing\AssertableInertia as Assert;

test('guest is redirected to login from module pages', function () {
    $this->get('/pendaftaran')->assertRedirect(route('login'));
    $this->get('/admin/users')->assertRedirect(route('login'));
});

test('registered user always gets the user role', function () {
    $this->post('/register', [
        'name' => 'Mahasiswa',
        'email' => 'mahasiswa@example.com',
        'password' => 'password',
        'password_confirmation' => 'password',
        'role' => 'admin',
    ]);

    expect(User::where('email', 'mahasiswa@example.com')->first()->isAdmin())->toBeFalse();
});

test('user cannot open the user management page', function () {
    $this->actingAs(User::factory()->create())
        ->get('/admin/users')
        ->assertForbidden();
});

test('admin can see the user list rendered by the module page', function () {
    $this->actingAs(User::factory()->admin()->create())
        ->get('/admin/users')
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page
            ->component('UserManagement::index', false)
            ->has('users.data', 1));
});

test('admin can create a user', function () {
    $this->actingAs(User::factory()->admin()->create())
        ->post('/admin/users', [
            'name' => 'User Baru',
            'email' => 'baru@example.com',
            'role' => 'user',
            'password' => 'password',
            'password_confirmation' => 'password',
        ])
        ->assertRedirect('/admin/users')
        ->assertSessionHas('success');

    $this->assertDatabaseHas('users', ['email' => 'baru@example.com', 'role' => 'user']);
});

test('admin cannot demote or delete their own account', function () {
    $admin = User::factory()->admin()->create();

    $this->actingAs($admin)
        ->put("/admin/users/{$admin->id}", [
            'name' => $admin->name,
            'email' => $admin->email,
            'role' => 'user',
        ])
        ->assertSessionHasErrors('role');

    $this->actingAs($admin)
        ->delete("/admin/users/{$admin->id}")
        ->assertSessionHas('error');

    $this->assertDatabaseHas('users', ['id' => $admin->id, 'role' => 'admin']);
});
```

Kode 48. UserManagementTest.

Pengujian modul Pendaftaran memakai Storage::fake('public') agar folder storage asli tidak terisi. Test memeriksa render halaman, endpoint data JSON, error 422 saat form kosong,

penyimpanan data beserta berkas, penolakan berkas zip, penggantian berkas lama, penghapusan data beserta berkas, dan error 404 untuk id yang tidak ada.

```php
<?php

use App\Models\User;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Inertia\Testing\AssertableInertia as Assert;
use Modules\Pendaftaran\Models\Daftar;

beforeEach(function () {
    Storage::fake('public');
    $this->actingAs(User::factory()->create());
});

function dataPendaftar(array $overrides = []): array
{
    return [
        'nama' => 'Budi Santoso',
        'asal_sekolah' => 'SMA Negeri 1',
        'prodi_tujuan' => 'Informatika',
        'email' => 'budi@example.com',
        'no_hp' => '081234567890',
        ...$overrides,
    ];
}

test('index renders the react page of the module', function () {
    $this->get('/pendaftaran')
        ->assertOk()
        ->assertInertia(fn (Assert $page) => $page->component('Pendaftaran::index',
false));
});

test('data endpoint returns registrations as json', function () {
    Daftar::factory()->count(3)->create();

    $this->getJson('/pendaftaran/data')->assertOk()->assertJsonCount(3);
});

test('store returns 422 when the form is empty', function () {
    $this->postJson('/pendaftaran', [])
        ->assertStatus(422)
        ->assertJsonValidationErrors(['nama', 'asal_sekolah', 'prodi_tujuan', 'email',
'no_hp', 'berkas']);
});

test('store saves the data and the uploaded file', function () {
    $this->postJson('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->create('rapor_budi.png', 100, 'image/png'),
    ]))->assertOk()->assertJson(['success' => 'Data berhasil disimpan.']);

    $daftar = Daftar::first();
    expect($daftar->nama_berkas)->toBe('rapor_budi.png');
    Storage::disk('public')->assertExists($daftar->berkas);
});

test('zip file is rejected', function () {
    $this->postJson('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->create('berkas.zip', 100),
    ]))->assertStatus(422)->assertJsonValidationErrors('berkas');
});
```

Kode 49. PendaftaranTest bagian 1.

```php

test('update replaces the old file', function () {
    $this->postJson('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->create('lama.png', 100, 'image/png'),
    ]));

    $daftar = Daftar::first();
    $oldPath = $daftar->berkas;

    $this->postJson("/pendaftaran/{$daftar->id}", dataPendaftar([
        '_method' => 'PUT',
        'prodi_tujuan' => 'Sistem Informasi',
        'berkas' => UploadedFile::fake()->create('surat_keterangan.pdf', 100,
'application/pdf'),
    ]))->assertOk();

    Storage::disk('public')->assertMissing($oldPath);
    expect($daftar->fresh()->prodi_tujuan)->toBe('Sistem Informasi');
});

test('destroy deletes the data and its file, unknown id returns 404', function () {
    $this->postJson('/pendaftaran', dataPendaftar([
        'berkas' => UploadedFile::fake()->create('rapor.png', 100, 'image/png'),
    ]));

    $daftar = Daftar::first();

    $this->deleteJson("/pendaftaran/{$daftar->id}")->assertOk();
    Storage::disk('public')->assertMissing($daftar->berkas);
    $this->assertDatabaseMissing('daftars', ['id' => $daftar->id]);

    $this->deleteJson('/pendaftaran/999')->assertNotFound();
});
```

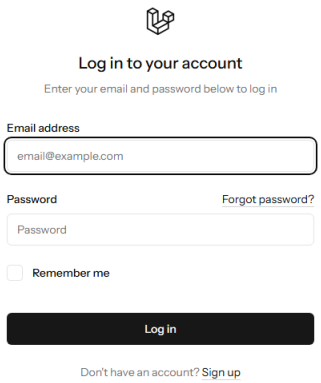
Kode 50. PendaftaranTest bagian 2.

D. Hasil dan Dokumentasi Program

Berikut tangkapan layar dari aplikasi yang sudah dijalankan pada alamat http://127.0.0.1:8010. Aplikasi berjalan dengan php artisan serve dan aset React hasil npm run build, sedangkan database diisi ulang dengan php artisan migrate:fresh --seed.

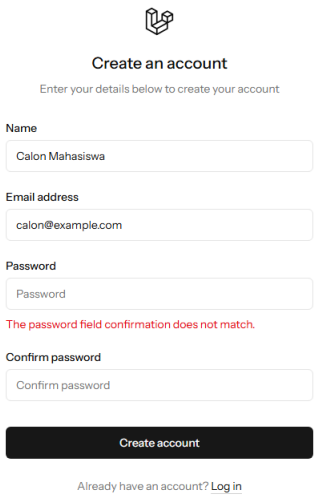
D.1 Autentikasi

Saat /pendaftaran dibuka tanpa login, middleware auth mengarahkan pengguna ke halaman login.



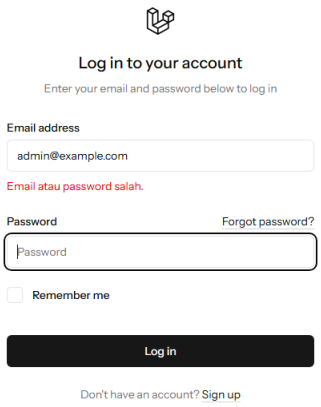
Gambar 1. Halaman login saat /pendaftaran dibuka tanpa login.

Halaman register menolak data yang tidak lengkap. Pesan galat muncul di bawah setiap input yang salah.



Gambar 2. Galat validasi pada form register.

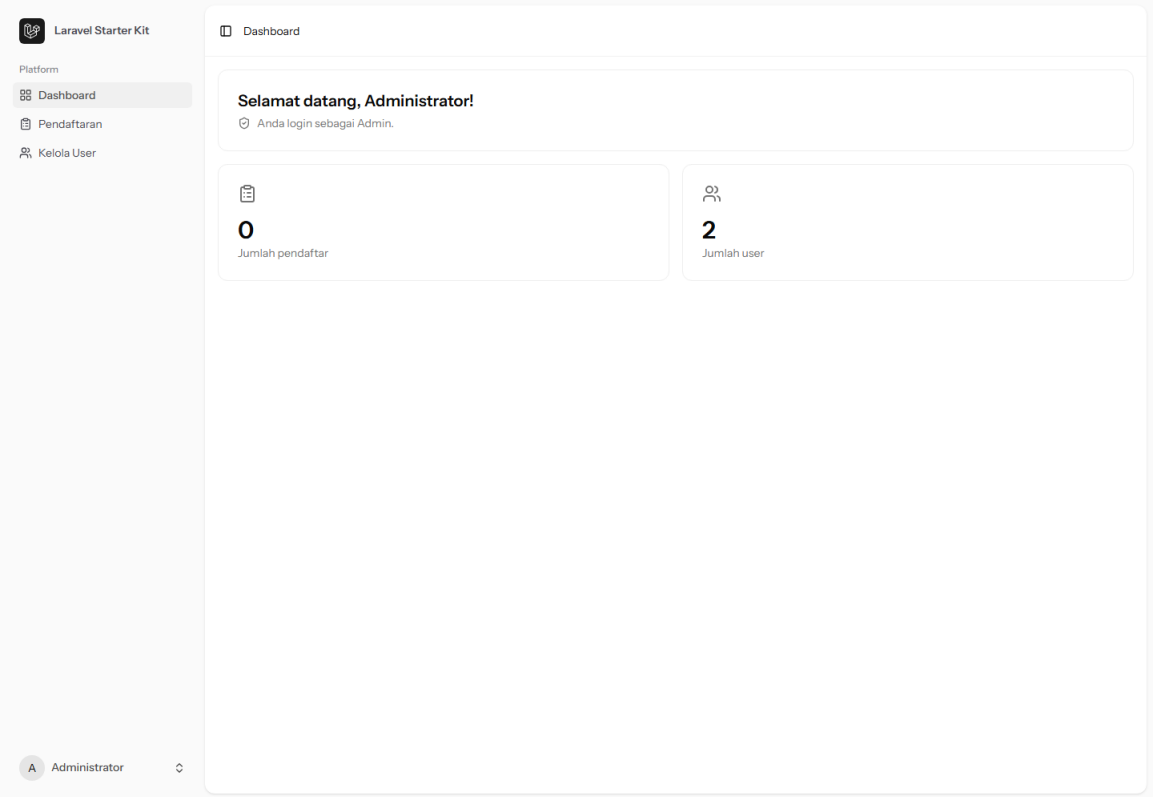
Login dengan password salah menghasilkan pesan Email atau password salah.



Gambar 3. Pesan galat saat password salah.

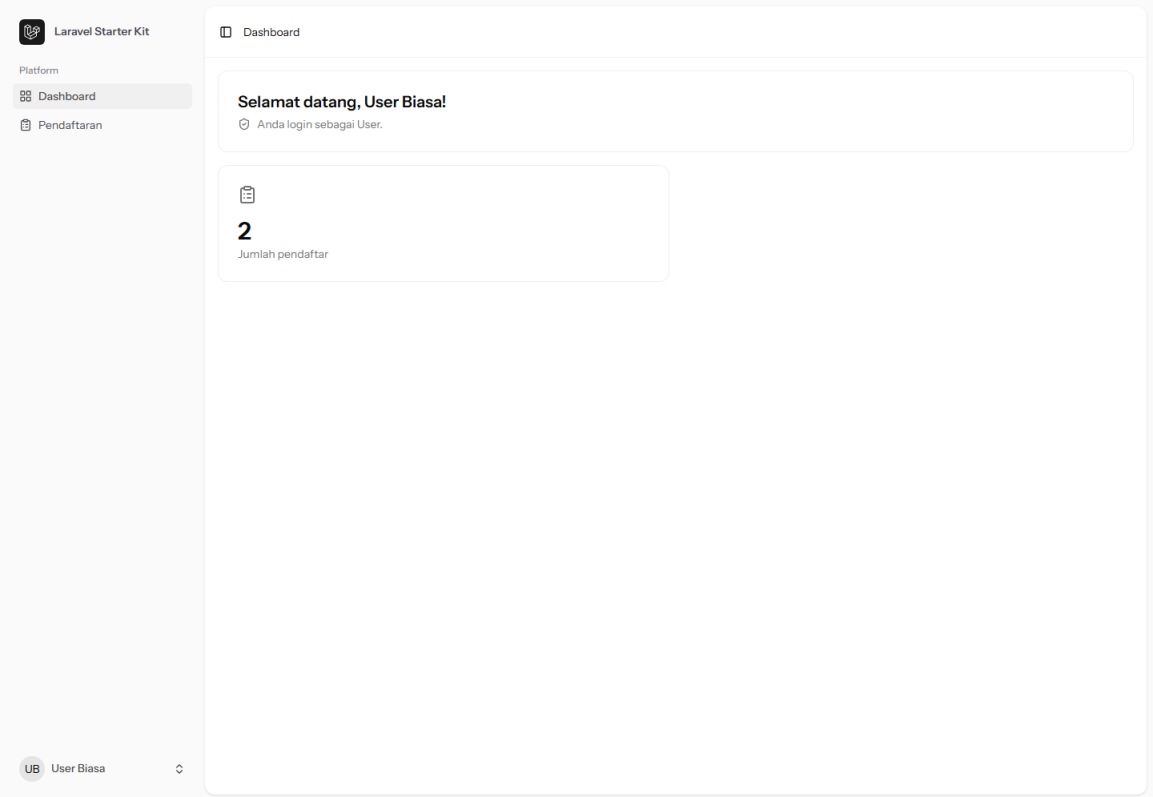
D.2 Dashboard dan Hak Akses

Dashboard admin menampilkan dua kartu, yaitu total pendaftar dan total user, serta menu Kelola User pada sidebar.



Gambar 4. Dashboard akun admin dengan dua kartu dan menu Kelola User.

Setelah login sebagai user, dashboard hanya menampilkan satu kartu, yaitu total pendaftar. Menu Kelola User tidak muncul karena isAdmin bernilai false.



Gambar 5. Dashboard akun user dengan satu kartu.

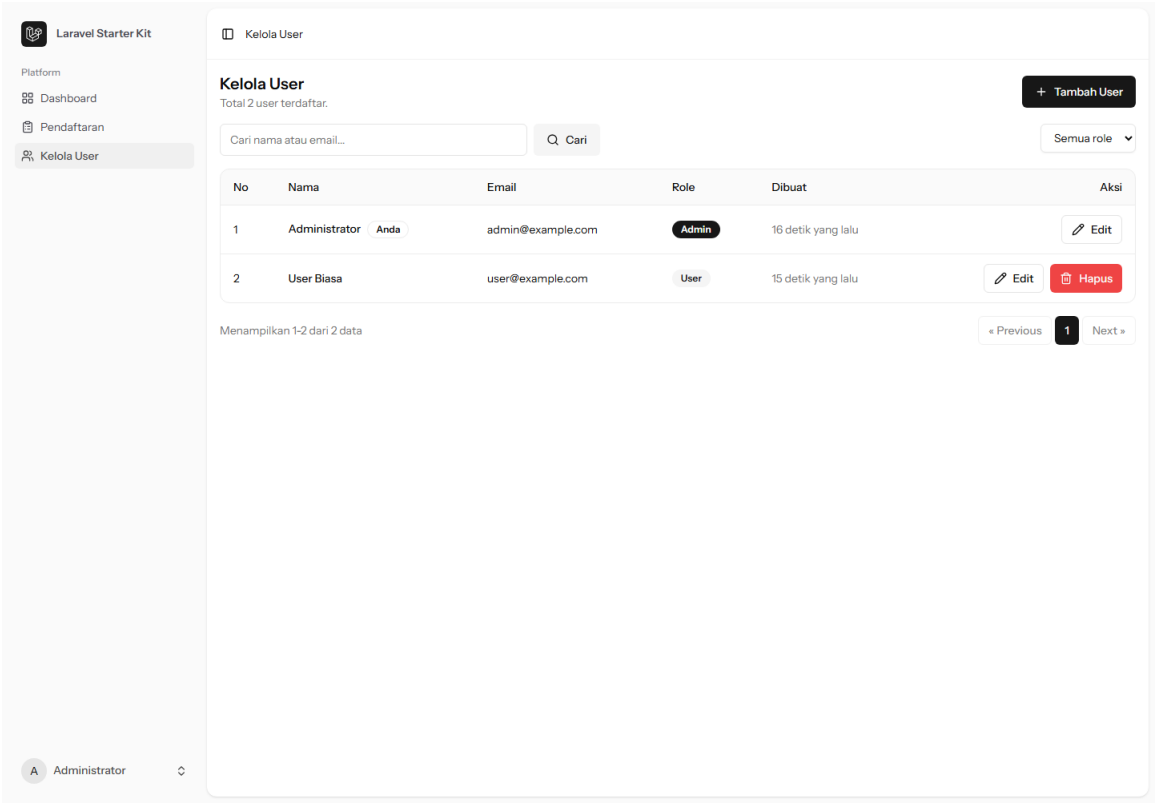
User biasa yang membuka alamat /admin/users secara langsung ditolak middleware role dengan kode 403.



Gambar 6. Error 403 saat user membuka halaman khusus admin.

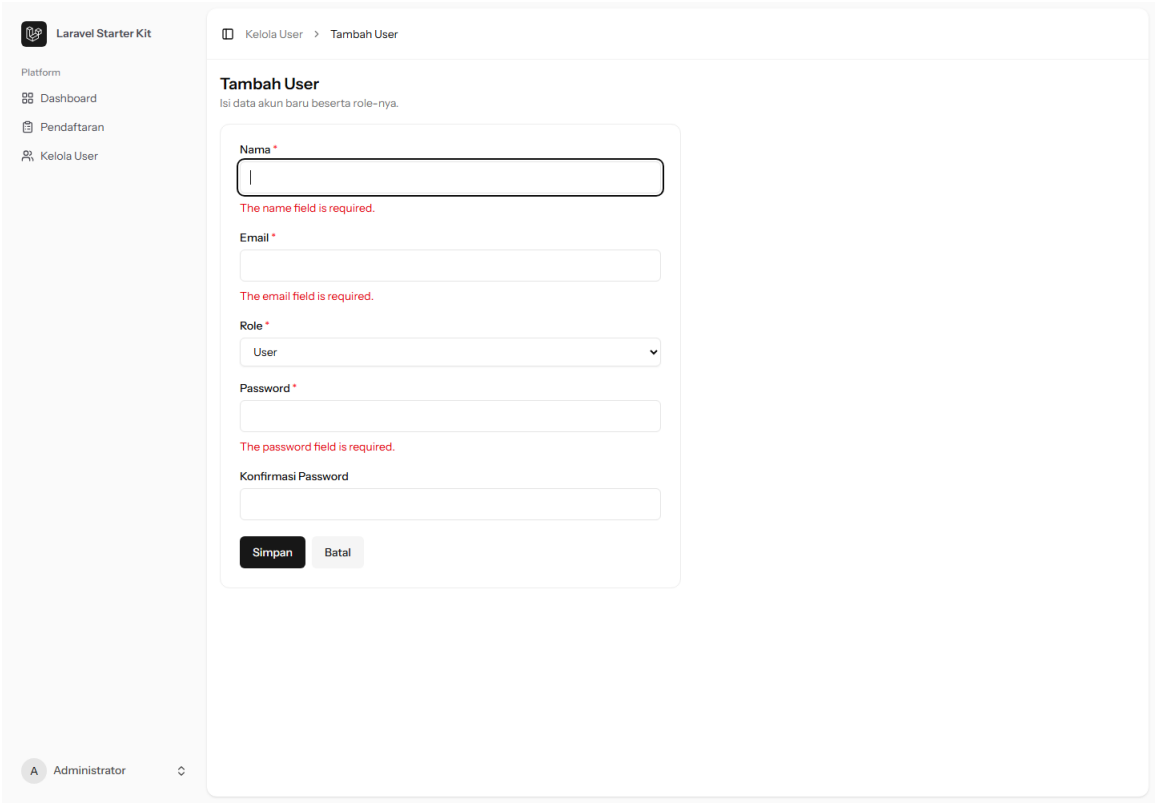
D.3 CRUD User oleh Admin

Halaman Kelola User menampilkan daftar user beserta role-nya. Akun yang sedang login diberi badge Anda dan tidak memiliki tombol Hapus.



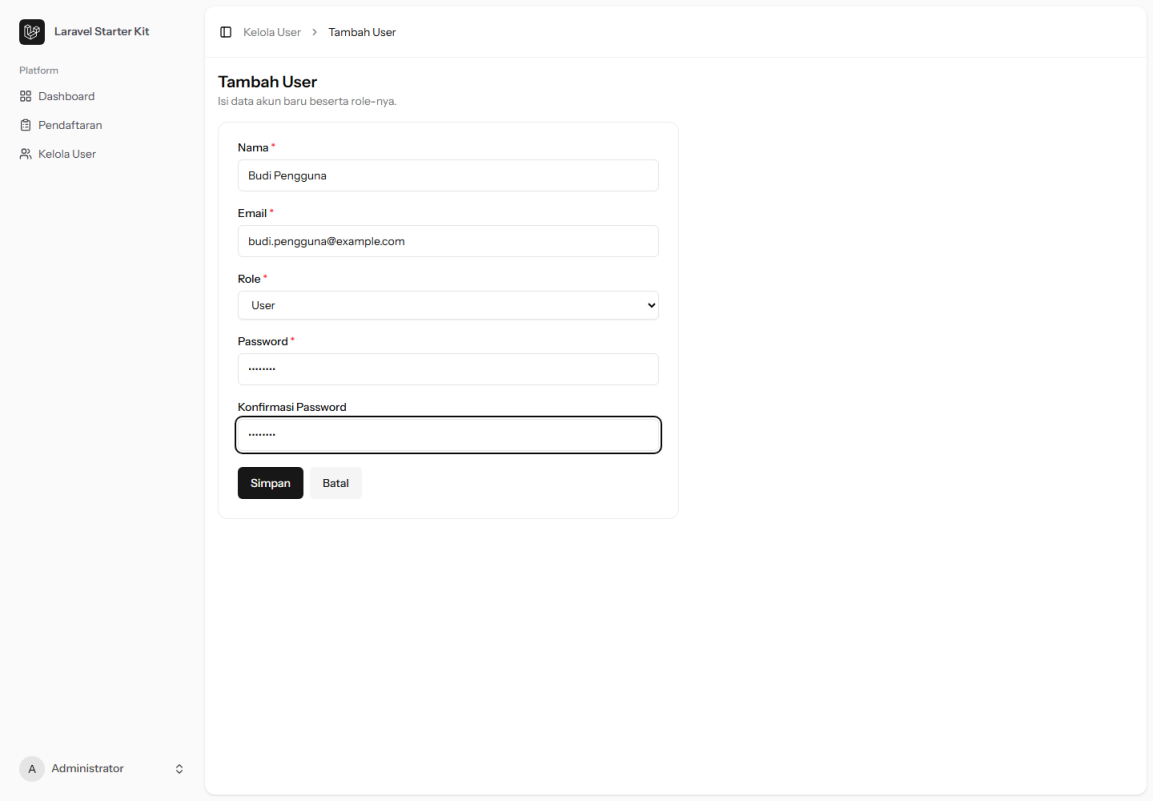
Gambar 7. Daftar user pada halaman Kelola User.

Form tambah user yang dikirim kosong menampilkan pesan galat di bawah setiap input.

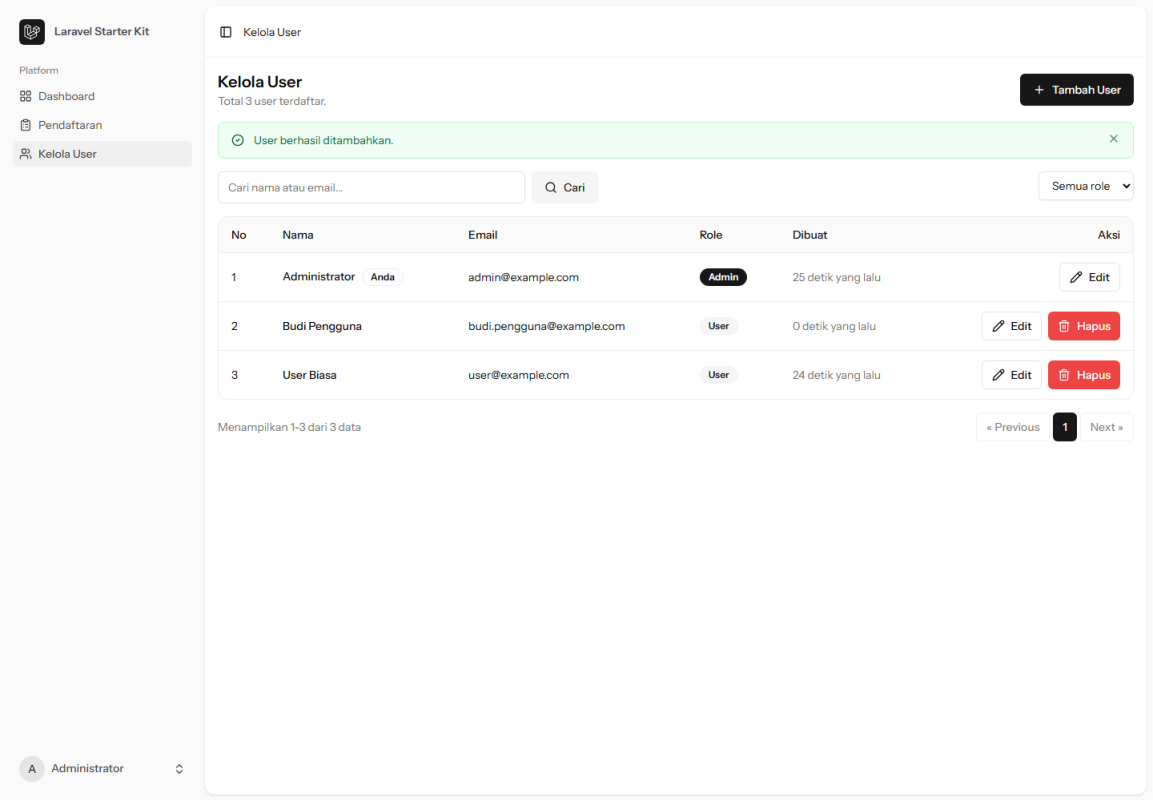


Gambar 8. Galat validasi pada form tambah user.

Setelah data diisi dan disimpan, aplikasi kembali ke daftar user disertai pesan berhasil. User baru langsung muncul pada tabel.

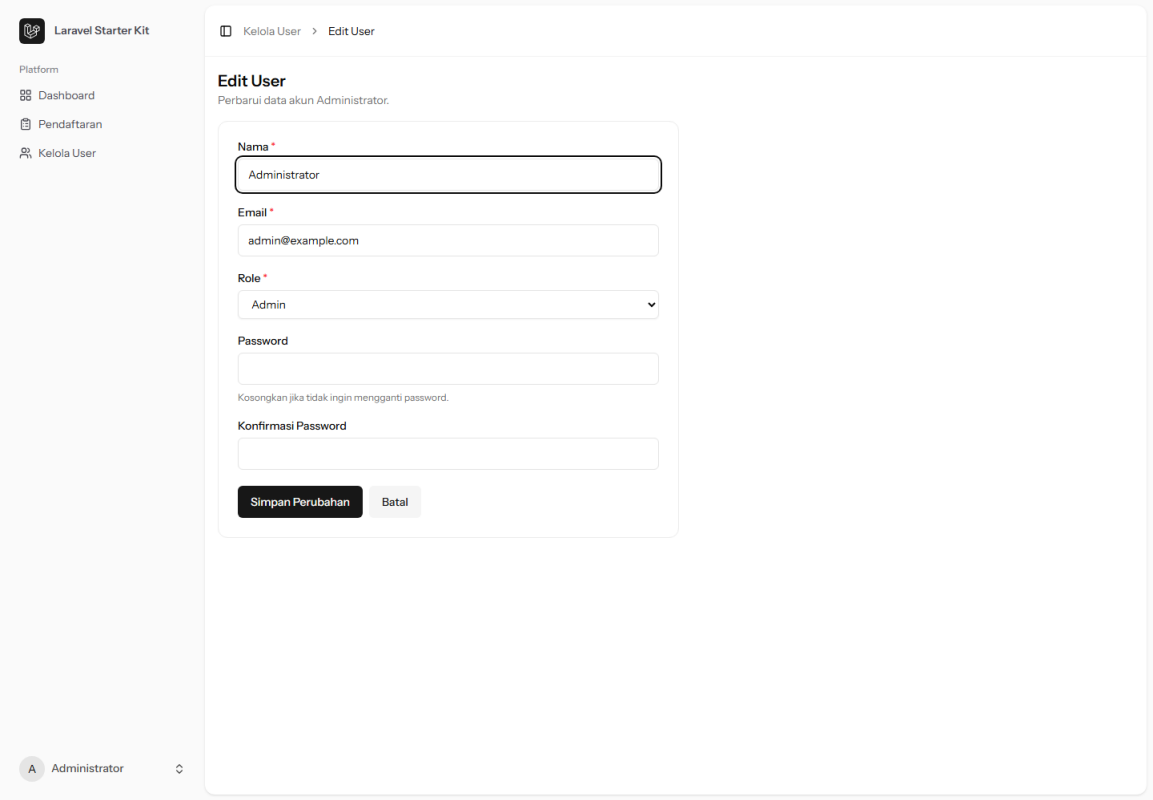


Gambar 9. Form tambah user yang sudah diisi.



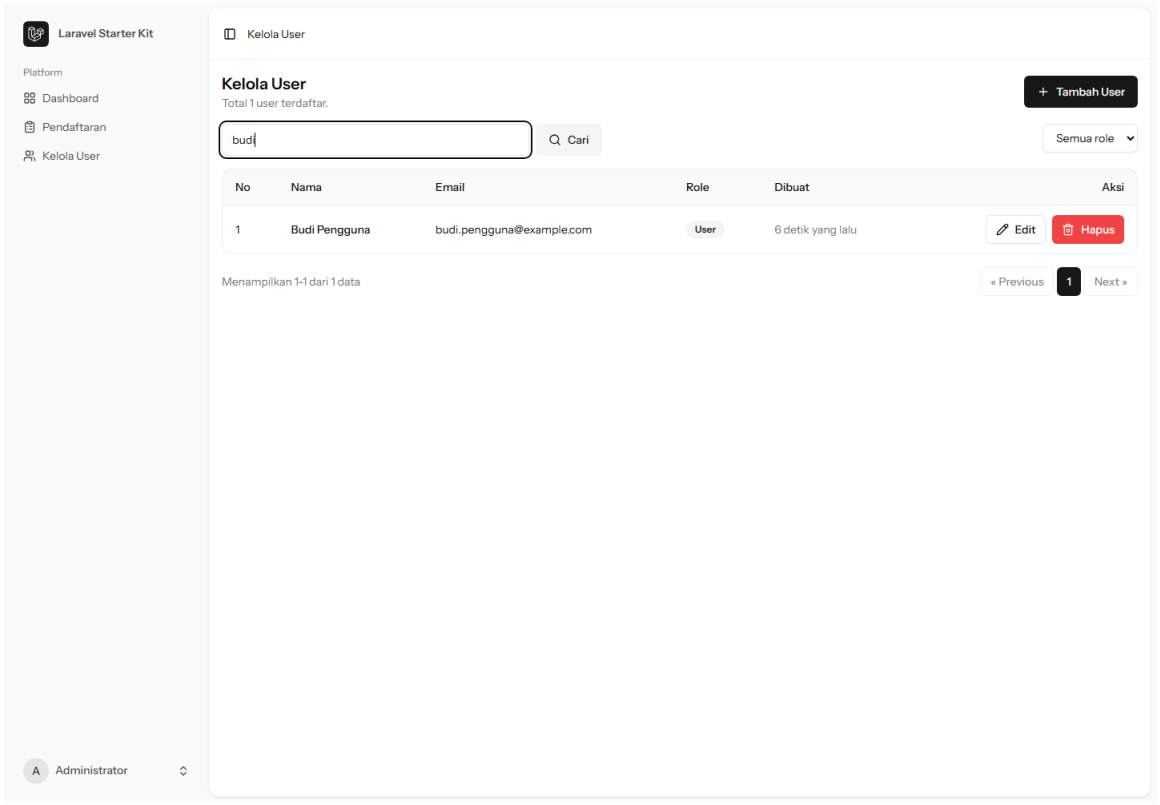
Gambar 10. Pesan berhasil setelah user ditambahkan.

Saat admin mengubah akunnya sendiri, pilihan role dibatasi sehingga role tidak dapat diturunkan menjadi user.



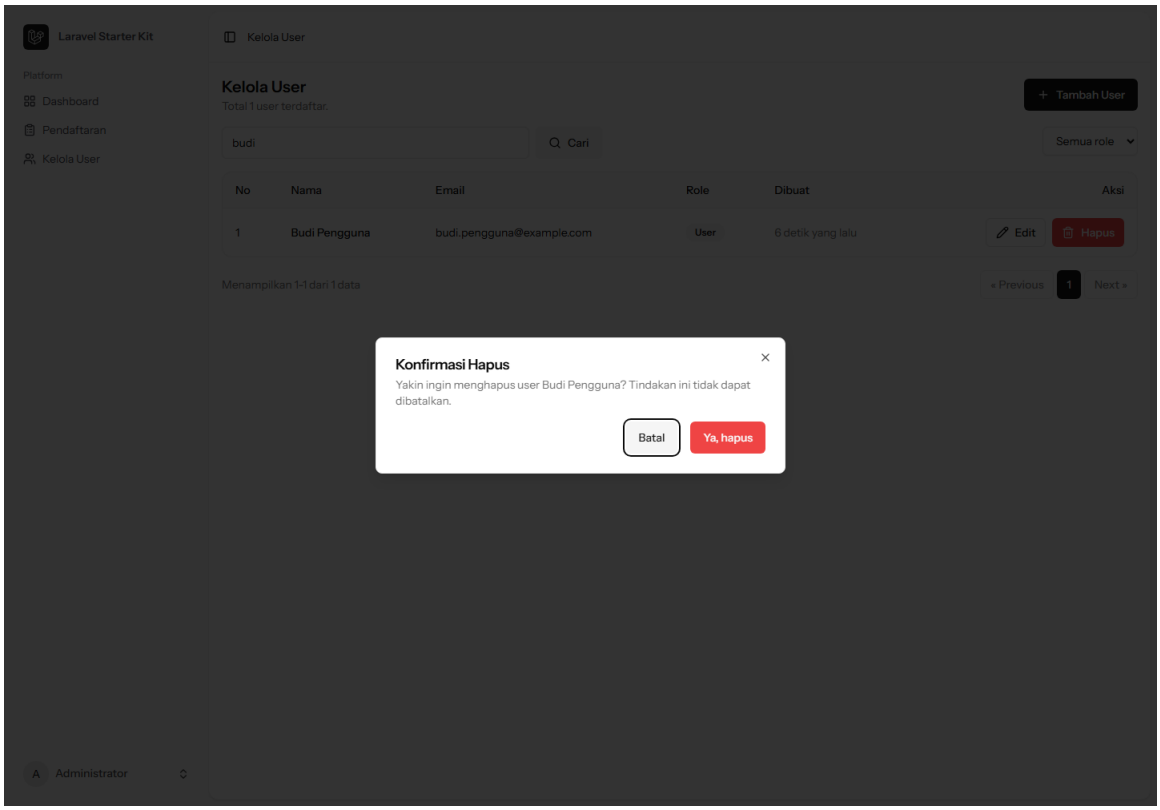
Gambar 11. Form edit akun admin sendiri dengan role terkunci.

Pencarian memfilter daftar berdasarkan nama atau email, dan kata kunci tampil pada URL sebagai query string.



Gambar 12. Pencarian user dengan kata kunci.

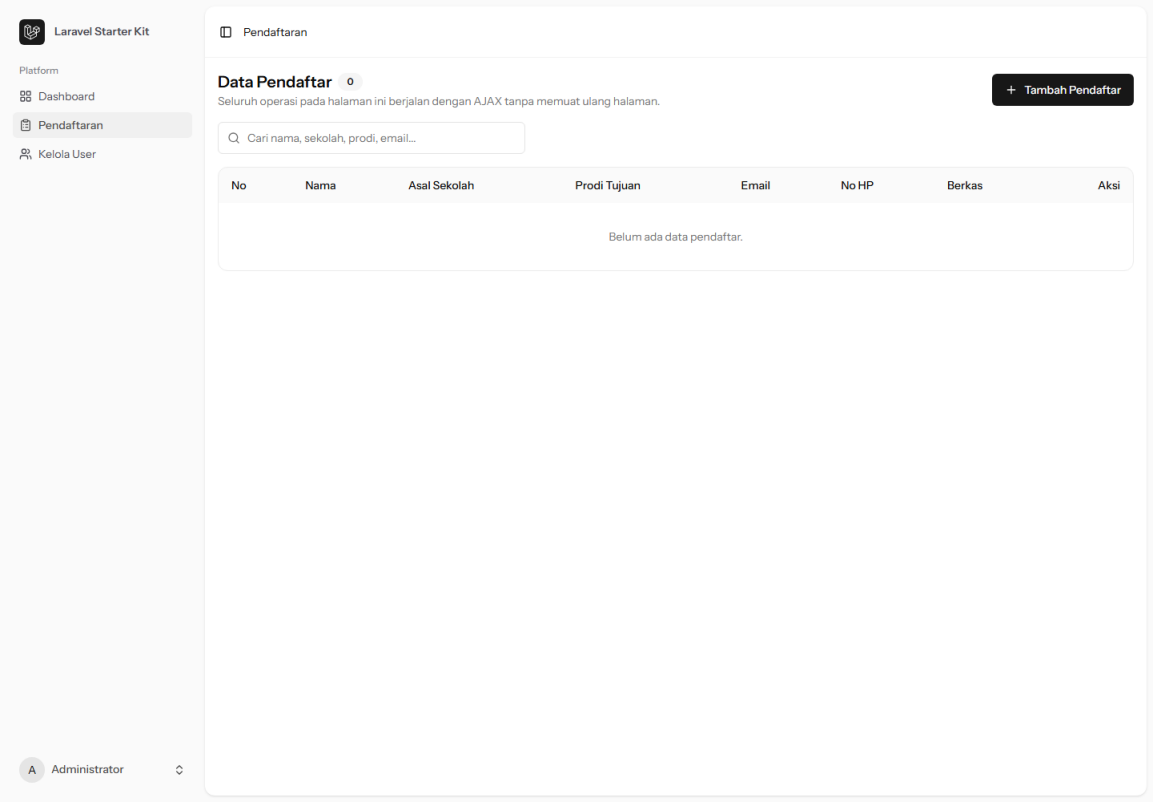
Penghapusan user memakai dialog konfirmasi bersama. Request DELETE baru dikirim setelah pengguna menekan Ya, hapus.



Gambar 13. Dialog konfirmasi sebelum user dihapus.

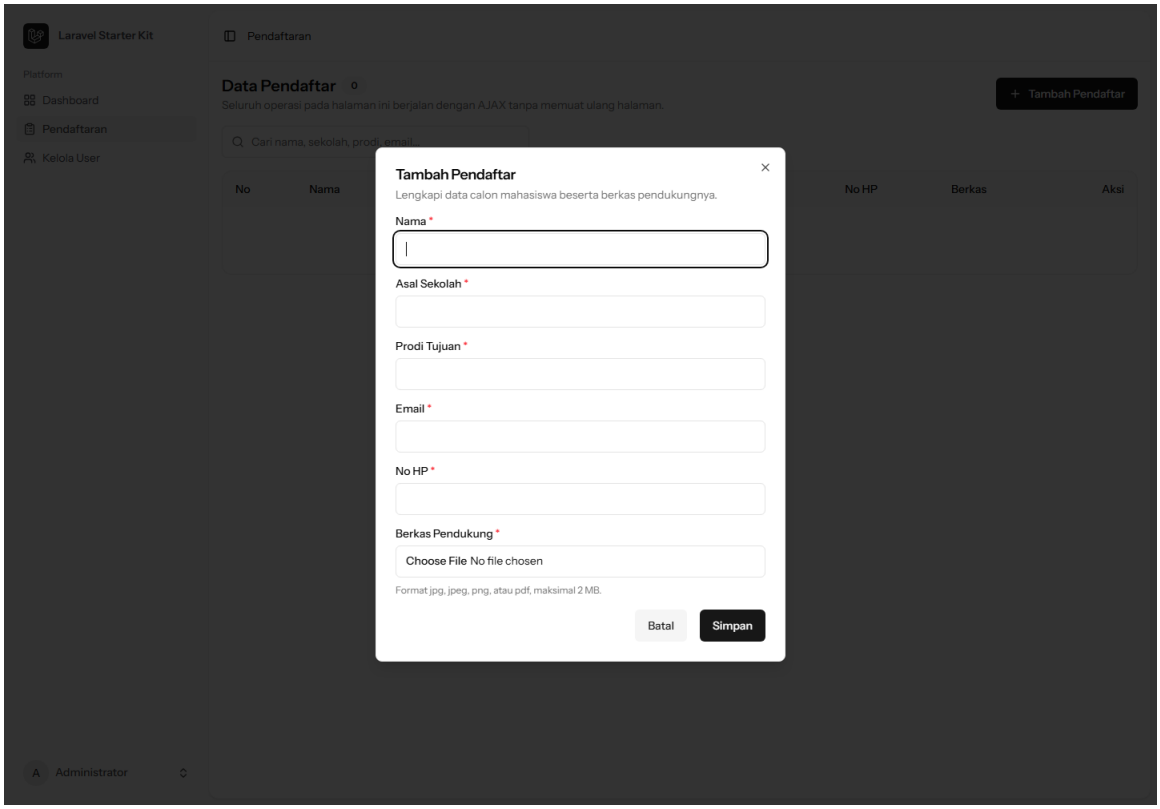
D.4 CRUD Pendaftaran dengan AJAX

Halaman pendaftaran dimuat sekali, lalu tabel diisi dari endpoint JSON. Saat belum ada data, tabel menampilkan pesan Belum ada data pendaftar.

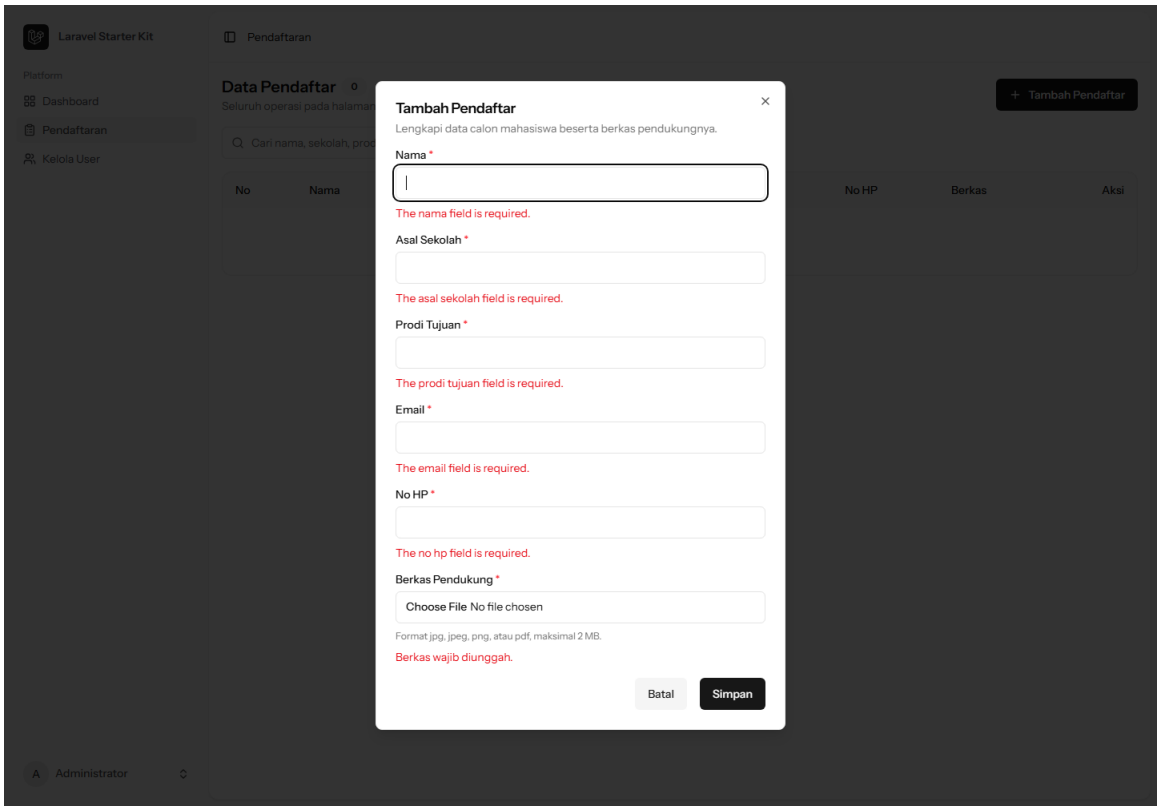


Gambar 14. Halaman Pendaftaran saat belum ada data.

Tombol Tambah Pendaftar membuka dialog form. Bila form dikirim kosong, server mengembalikan 422 dan pesan galat tampil di bawah setiap input tanpa menutup dialog.

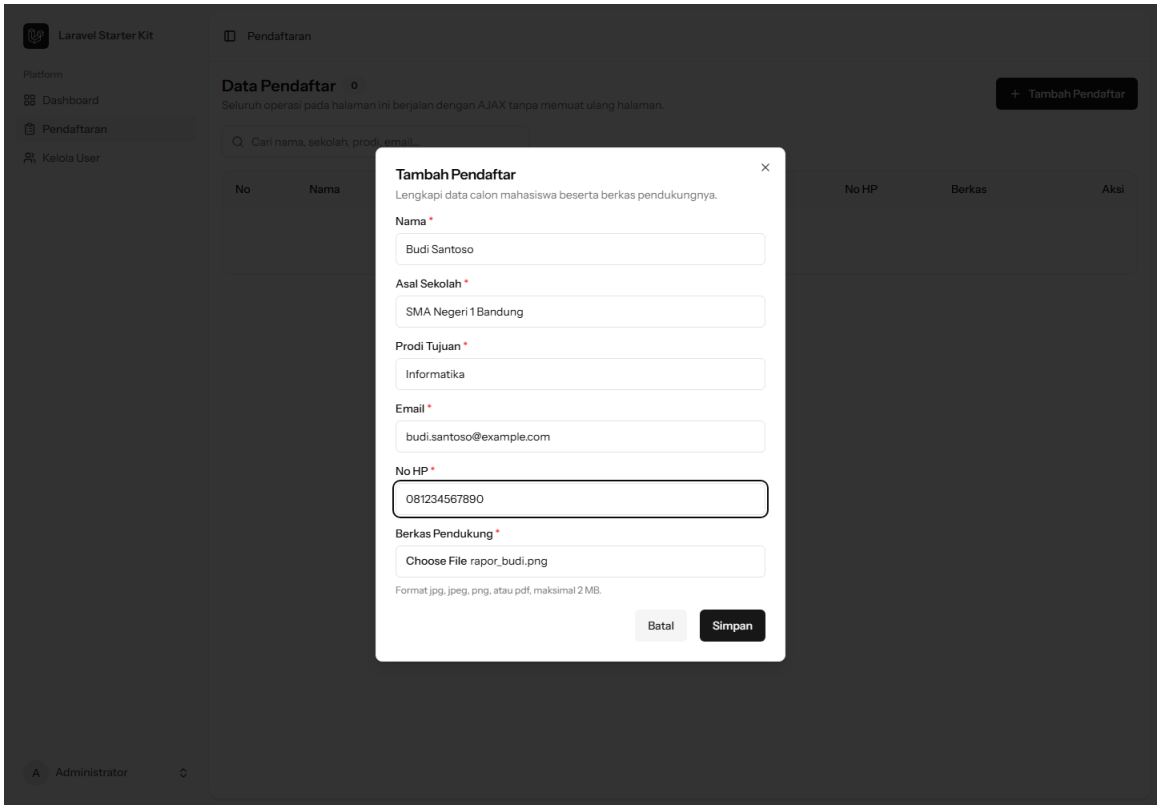


Gambar 15. Dialog tambah pendaftar dalam keadaan kosong.

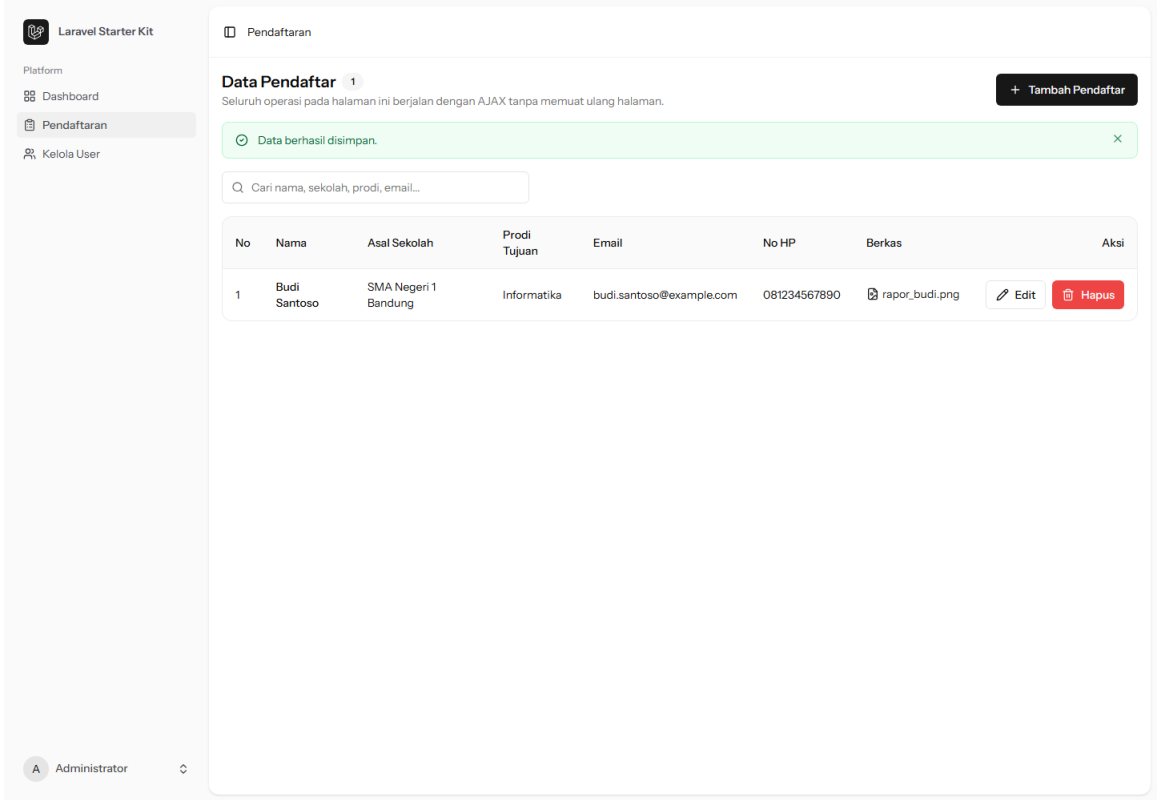


Gambar 16. Respons 422 saat form dikirim kosong.

Setelah seluruh data dan berkas diisi lalu disimpan, dialog tertutup, tabel diperbarui tanpa memuat ulang halaman, dan pesan berhasil muncul.

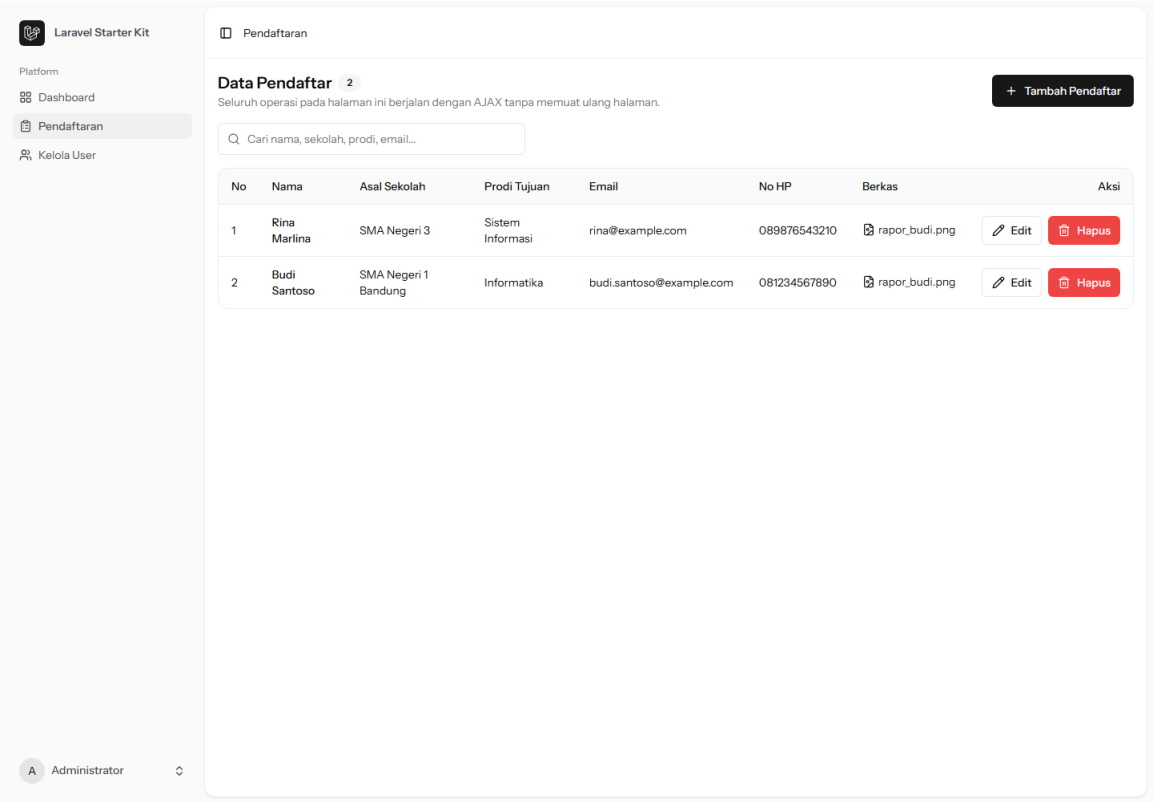


Gambar 17. Dialog tambah pendaftar yang sudah diisi beserta berkas.



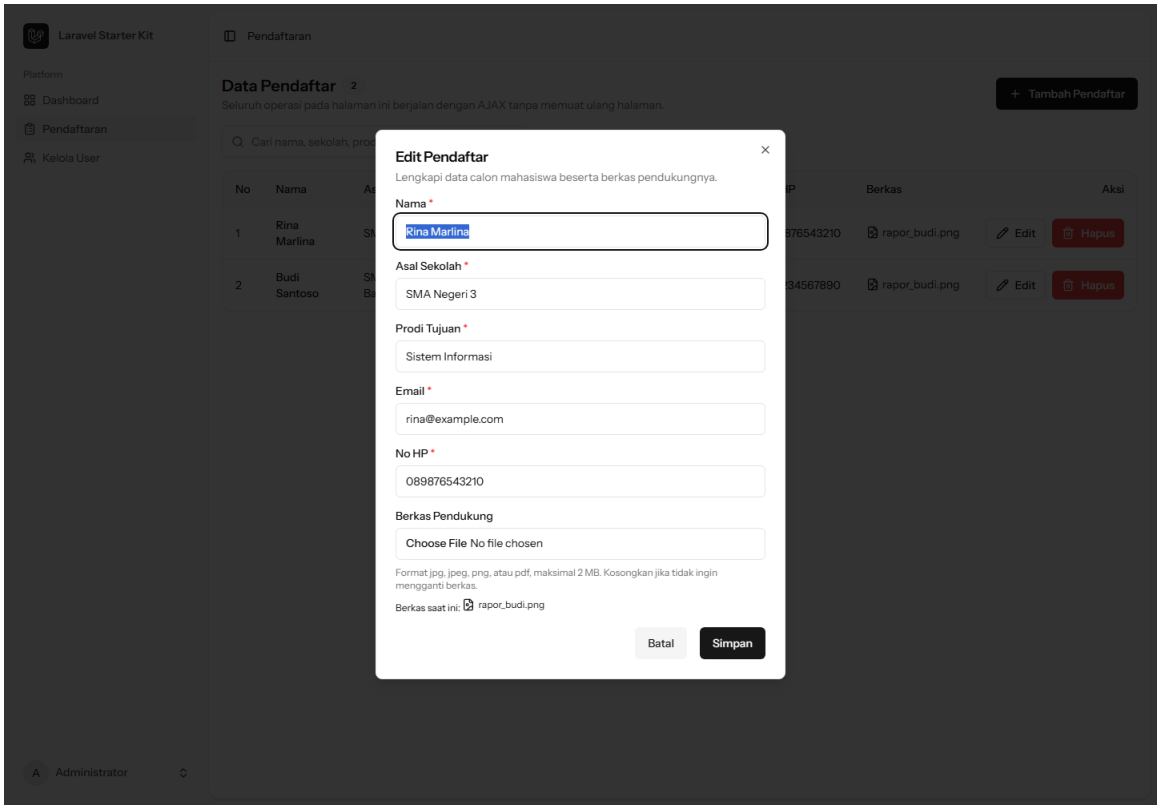
Gambar 18. Pesan berhasil setelah data disimpan lewat AJAX.

Menambah data kedua tidak memuat ulang halaman. Jumlah data pada badge bertambah dan tabel langsung menampilkan dua baris.



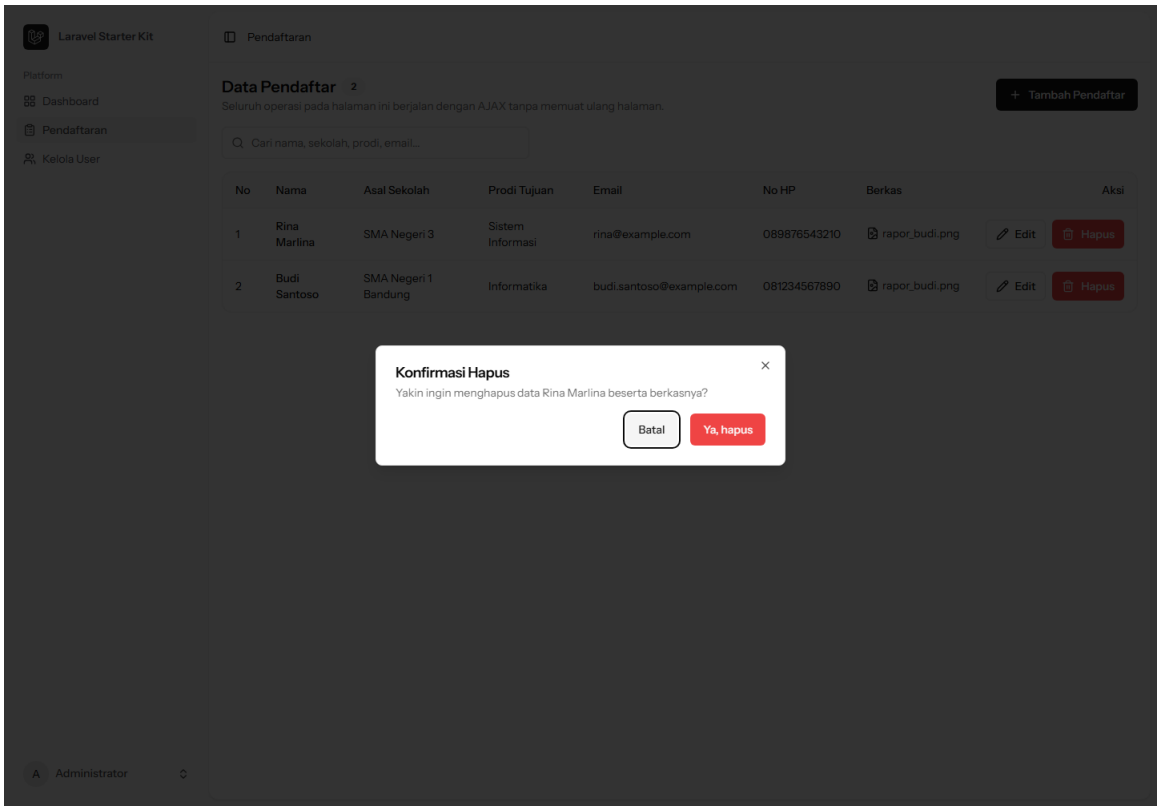
Gambar 19. Daftar pendaftar setelah dua data ditambahkan.

Dialog edit mengambil data lama lewat AJAX dan menampilkan tautan berkas saat ini. Berkas boleh dikosongkan jika tidak ingin menggantinya.



Gambar 20. Dialog edit dengan data lama dan tautan berkas saat ini.

Penghapusan data memakai dialog konfirmasi. Setelah dikonfirmasi, baris hilang dari tabel tanpa berpindah halaman.



Gambar 21. Dialog konfirmasi sebelum data dihapus.

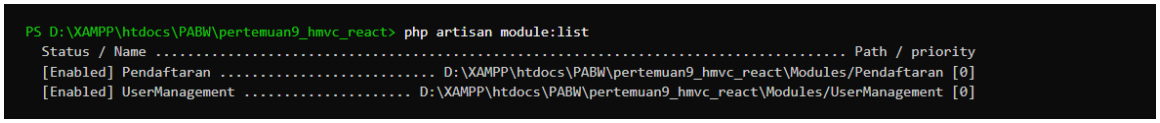
D.5 Bukti Respons JSON dan Berkas

Endpoint /pendaftaran/data mengembalikan array JSON berisi seluruh pendaftar beserta berkas_url. Data inilah yang diambil oleh fungsi loadData() pada halaman React.



Gambar 22. Respons JSON dari endpoint /pendaftaran/data.

Daftar modul diperiksa dengan php artisan module:list. Kedua modul berstatus Enabled sehingga route-nya dimuat oleh Laravel.



Gambar 23. Hasil php artisan module:list.

Seluruh route diperiksa dengan php artisan route:list --except-vendor. Terlihat enam route admin/users dari modul UserManagement dan enam route pendaftaran dari modul Pendaftaran.

```
PS D:\XAMPP\htdocs\PABW\pertemuan9_hmvc_react> php artisan route:list --except-vendor
  GET|HEAD       / ..................................................................... home › routes/web.php:8
  GET|HEAD       admin/users admin.users.index › Modules\UserManagement\Http\Controllers\UserManagementController@in…
  POST           admin/users admin.users.store › Modules\UserManagement\Http\Controllers\UserManagementController@st…
  GET|HEAD       admin/users/create admin.users.create › Modules\UserManagement\Http\Controllers\UserManagementContr…
  PUT|PATCH      admin/users/{user} admin.users.update › Modules\UserManagement\Http\Controllers\UserManagementContr…
  DELETE         admin/users/{user} admin.users.destroy › Modules\UserManagement\Http\Controllers\UserManagementCont…
  GET|HEAD       admin/users/{user}/edit admin.users.edit › Modules\UserManagement\Http\Controllers\UserManagementCo…
  GET|HEAD       dashboard ..................................................... dashboard › DashboardController
  GET|HEAD       pendaftaran ... pendaftaran.index › Modules\Pendaftaran\Http\Controllers\PendaftaranController@index
  POST           pendaftaran ... pendaftaran.store › Modules\Pendaftaran\Http\Controllers\PendaftaranController@store
  GET|HEAD       pendaftaran/data pendaftaran.data › Modules\Pendaftaran\Http\Controllers\PendaftaranController@getD…
  PUT            pendaftaran/{daftar} pendaftaran.update › Modules\Pendaftaran\Http\Controllers\PendaftaranControlle…
  DELETE         pendaftaran/{daftar} pendaftaran.destroy › Modules\Pendaftaran\Http\Controllers\PendaftaranControll…
  GET|HEAD       pendaftaran/{daftar}/edit pendaftaran.edit › Modules\Pendaftaran\Http\Controllers\PendaftaranContro…
  ANY            settings ..................................................... Illuminate\Routing › RedirectController
  GET|HEAD       settings/appearance ..................................... appearance › routes/settings.php:18
  GET|HEAD       settings/password ................................ password.edit › Settings\PasswordController@edit
  PUT            settings/password ............................. password.update › Settings\PasswordController@update
  GET|HEAD       settings/profile ..................................... profile.edit › Settings\ProfileController@edit
  PATCH          settings/profile ............................... profile.update › Settings\ProfileController@update
  DELETE         settings/profile ............................... profile.destroy › Settings\ProfileController@destroy
                                                                                                  Showing [21] routes
```

Gambar 24. Hasil php artisan route:list --except-vendor.

Pemeriksaan terakhir dilakukan pada folder penyimpanan. Nama berkas di disk berupa hash acak hasil store(), sedangkan database hanya menyimpan path dan nama asli berkas.

```
PS D:\XAMPP\htdocs\PABW\pertemuan9_hmvc_react> dir storage\app\public\berkas_pendaftaran
 Volume in drive D is DATA
 Volume Serial Number is 120C-C661
 Directory of D:\XAMPP\htdocs\PABW\pertemuan9_hmvc_react\storage\app\public\berkas_pendaftaran
09/11/2026  13:17    <DIR>          .
09/11/2026  11:11    <DIR>          ..
09/11/2026  13:17                67 0yOh8gZ75g1P6ICyo98qWU40IFS5sm21jY1w6Iru.png
09/11/2026  13:17                67 aHHH8PWbOs1S3cQaFSbVO7S7spNLARlefJ5zxv5Q.png
09/11/2026  11:11            18.441 pZ3Z4RKDkrJ4syKDEWQnPD7Yxra7U6fqkfPY1hK1.jpg
               3 File(s)         18.575 bytes
               2 Dir(s)  24.494.239.744 bytes free
```

Gambar 25. Isi folder storage/app/public/berkas_pendaftaran.

E. Analisis dan Pembahasan

E.1 Cara Kerja HMVC dengan Halaman React

HMVC membagi aplikasi menjadi modul yang masing-masing memiliki controller, request, model, migration, route, dan halaman React sendiri. Pada project ini modul Pendaftaran dan UserManagement berada di folder Modules, sedangkan autentikasi memakai Fortify di app. Namespace kelas mengikuti nama modul, misalnya Modules\Pendaftaran\Http\Controllers\PendaftaranController.

Halaman React milik modul tidak berada di resources/js/pages, melainkan di Modules/*/resources/js/pages. Karena itu Inertia perlu tahu cara menemukannya. Fungsi resolve pada app.tsx membaca format NamaModul::halaman, memecahnya, lalu memuat file yang sesuai lewat resolvePageComponent. Vite sendiri mengumpulkan seluruh halaman karena import.meta.glob mencakup folder modul.

E.2 Perbedaan Authentication dan Authorization

Authentication menjawab pertanyaan siapa penggunanya, sedangkan authorization menjawab apa yang boleh dilakukannya. Pada praktikum ini authentication sepenuhnya ditangani Fortify beserta halaman React-nya, jadi saya tidak menulis LoginController maupun RegisterController seperti pada Modul 8. Authorization ditangani middleware EnsureUserHasRole yang membaca kolom role.

Pemisahan keduanya penting. Semua pengguna yang sudah login lolos middleware auth, tetapi hanya admin yang lolos middleware role:admin. Menu Kelola User pada sidebar hanya menyembunyikan tautan; perlindungan sebenarnya tetap di server, sehingga user yang mengetik /admin/users langsung tetap menerima error 403.

E.3 Middleware Role dan Pembatasan Akun Sendiri

Middleware menerima parameter role yang dipisahkan koma. Laravel mengirim bagian setelah titik dua sebagai argumen variadik, lalu middleware memanggil hasRole(). Pemeriksaan tambahan saya taruh pada UpdateUserRequest dan method destroy. Admin yang mengubah akunnya sendiri hanya bisa memilih role admin, dan admin tidak bisa menghapus akunnya sendiri. Aturan ini mencegah admin kehilangan akses secara tidak sengaja.

E.4 Perbedaan CRUD Inertia dan CRUD AJAX

CRUD UserManagement bekerja secara Inertia. Form dikirim lewat useForm, server memproses lalu mengembalikan halaman Inertia atau redirect, dan Inertia mengganti komponen halaman tanpa memuat ulang seluruh browser. Data selalu dikirim ulang oleh server sebagai props, dan error validasi otomatis masuk ke objek errors milik useForm. Pesan berhasil dibagikan sebagai shared props flash.

CRUD Pendaftaran bekerja secara AJAX. Pengiriman form dikirim lewat fetch dan controller mengembalikan JSON. Tabel diperbarui dengan memanggil loadData() tanpa memuat ulang halaman, dan pesan ditampilkan dari respons JSON. Akibatnya, posisi scroll dan isi kotak pencarian tetap terjaga. Konsekuensinya, React harus menangani sendiri status 422 dan mengisi pesan galat ke setiap input.

E.5 FormData dan Penyimpanan Berkas

Upload berkas tidak bisa memakai JSON biasa karena JSON tidak dapat membawa file. FormData dipakai karena objek itulah yang bisa membawa isi input type=file. Pada helper http.ts, header Content-Type sengaja tidak diisi agar browser otomatis memakai

multipart/form-data beserta boundary. Untuk edit, request tetap dikirim sebagai POST dengan tambahan _method PUT karena PHP hanya membaca file pada request POST.

Berkas disimpan di disk public dengan store(), sedangkan database hanya menyimpan path dan nama asli. Pemisahan ini membuat tabel tetap ringan dan berkas mudah diganti. Saat edit tanpa memilih berkas baru, kunci berkas dibuang dari data yang divalidasi sehingga path lama tetap dipakai. Saat berkas diganti atau data dihapus, berkas lama ikut dihapus dari storage agar tidak ada file sampah.

E.6 Kendala dan Penyelesaian

1. Kelas modul sempat tidak ditemukan karena autoload belum diperbarui setelah modul dibuat. Perbaikannya menjalankan composer dump-autoload.

2. Halaman modul sempat tampil dengan pesan Unable to locate file in Vite manifest karena app.blade.php ikut memuat berkas halaman secara langsung, padahal halaman modul berada di luar resources/js/pages. Perbaikannya cukup memuat app.tsx pada @vite, karena seluruh halaman sudah di-resolve oleh Inertia melalui import.meta.glob.

3. Kelas middleware HandleAppearance sempat tidak ditemukan karena berkasnya hilang dari folder app/Http/Middleware padahal masih dirujuk bootstrap/app.php. Saya membuat ulang kelas tersebut agar request web dapat diproses.

4. Halaman login dan register sempat gagal dirender karena FortifyServiceProvider belum mendaftarkan view Inertia. Saya menambahkan Fortify::loginView, registerView, dan view lain beserta konfigurasi rate limiting pada provider tersebut.

5. Koneksi database ditolak dengan pesan SQLSTATE[HY000] [2002]. Penyebabnya layanan MariaDB pada XAMPP belum berjalan. Setelah layanan dinyalakan, aplikasi kembali normal.

6. Perintah npm sempat diblokir kebijakan eksekusi PowerShell. Saya menjalankannya melalui cmd agar skrip npm dapat dieksekusi.

7. Pengujian upload sempat gagal karena ekstensi GD tidak aktif pada PHP CLI, sehingga UploadedFile::fake()->image() tidak bisa membuat gambar. Saya menggantinya dengan UploadedFile::fake()->create() yang menyertakan tipe mime, sehingga test tetap menguji validasi berkas tanpa bergantung pada GD.

8. Enam test bawaan starter kit untuk email verification dan password confirmation gagal karena kedua fitur itu memang tidak diaktifkan pada project ini. Test tersebut saya tandai skip dengan alasan yang jelas, bukan dihapus.

E.7 Catatan Perbedaan dengan Modul

Ada beberapa perbedaan antara hasil praktikum ini dan contoh pada modul. Saya mencatatnya agar tidak menimbulkan salah paham saat laporan diperiksa.

1. Nama project adalah pertemuan9_hmvc_react dengan database hmvc_react, sesuai nama pada modul.

2. Seluruh tampilan memakai React Starter Kit dengan Inertia, bukan Blade dan jQuery seperti Modul 8.

3. Kolom data pendaftar mencakup nama, asal_sekolah, prodi_tujuan, email, no_hp, berkas, dan nama_berkas, dengan email sebagai kolom unik.

4. Sebagian pesan galat validasi bawaan Laravel pada form AJAX masih berbahasa Inggris, sedangkan pesan untuk no_hp dan berkas sudah berbahasa Indonesia lewat method messages().

5. Aplikasi diuji pada alamat http://127.0.0.1:8010, bukan http://127.0.0.1:8000, karena port 8000 sedang dipakai proses lain.

F. Kesimpulan

1. Arsitektur HMVC berhasil diterapkan dengan dua modul, yaitu Pendaftaran dan UserManagement, yang masing-masing memiliki controller, request, model, migration, route, dan halaman React sendiri di dalam folder Modules.

2. Halaman React di dalam modul dapat dimuat melalui Inertia setelah fungsi resolve pada app.tsx diatur dengan format NamaModul::halaman, dan Tailwind CSS ikut membuat class yang dipakai modul setelah baris @source ditambahkan.

3. Autentikasi memakai fitur bawaan Fortify, sedangkan pembagian hak akses dilakukan dengan kolom role, enum UserRole, middleware role, dan shared props yang membedakan admin dan user.

4. Middleware role dan aturan tambahan pada Form Request berhasil membatasi halaman khusus admin, termasuk mencegah admin menurunkan role atau menghapus akunnya sendiri.

5. CRUD UserManagement berjalan secara Inertia dengan form useForm, pesan berhasil dari shared props flash, dan error validasi yang otomatis tampil di bawah setiap input.

6. CRUD Pendaftaran berjalan dengan fetch dan FormData, menampilkan error 422 di bawah input, memperbarui tabel tanpa memuat ulang halaman, serta mengelola upload berkas beserta penggantian dan penghapusannya.

7. Pengujian otomatis Pest lulus seluruhnya untuk pengujian modul, sedangkan pengujian authentication bawaan starter kit untuk email verification dan password confirmation dilewati karena fiturnya tidak diaktifkan.

Dari praktikum ini saya memahami bahwa HMVC dengan React memisahkan tanggung jawab antarmodul tanpa kehilangan pola Laravel, bahwa authentication dan authorization adalah dua hal berbeda, dan bahwa CRUD Inertia maupun AJAX sama-sama menuntut penanganan error validasi serta pesan yang teliti di sisi klien.

G. Link Repository Github

https://github.com/Dryhus21/PABW.git